



SOFTWARE ARCHITECTURE (CO3017)

CC01 - Assignment Report

Intelligent Restaurant Management System (IRMS)

Lecturer: PhD Truong Thi Thai Minh
Students: Bui Quoc Thai - 2353086
Nguyen Vinh Anh Quan - 2353007
Le Thi Phuong Thao - 2252757
Huynh Ngoc Van - 2252898
Nguyen Thuy Tien - 2252806

Ho Chi Minh City, April 2026

The complete source code is available at [GitHub Repo](#).

Workload Distribution

No.	Full Name	Student ID	Tasks	Completion
1	Le Thi Phuong Thao	2252757	Project context (overview, scope & objectives, functional & non-functional requirements), ERD, Class Diagram, Architecture decisions, Backend (Ordering module), Frontend integration, apply SOLID	100%
2	Nguyen Thuy Tien	2252806	Design Principles, Package diagram, Deployment diagram, frontend (Kitchen Staff UI)	100%
3	Huynh Ngoc Van	2252898	Architecture Characteristics, Module View (Package diagram), Allocation View (Deployment diagram) and implement the frontend for Front Staff UI - Order Feature	100%
4	Nguyen Vinh Anh Quan	2353007	Project context (functional & non-functional requirements, actors & stakeholders), Architecture Style, Class Diagram, Component Diagram, Backend (KDS module)	100%
5	Bui Quoc Thai	2353086	Backend (KDS module), Class Diagram, Component Diagram, apply SOLID	100%

Contents

1	Context of the IRMS	3
1.1	Project Overview	3
1.2	Objectives & Scope	3
1.3	Stakeholders and Actors	3
1.4	Functional Requirements	4
1.5	Non-functional Requirements	5
2	Software Architecture for the IRMS	5
2.1	Architecture Characteristics	5
2.2	Architecture Style	6
2.3	Architecture Documents	7
2.3.1	Module View	7
2.3.2	Component Diagram	26
2.3.3	Allocation View	29
2.3.4	Entity-Relationship Diagram	31
2.4	Architecture Decisions	32
2.5	Design Principles	34
3	Core Implementation	35
3.1	Architectural Approach	35
3.2	Backend Module Structure	36
3.2.1	Order Module	36
3.2.2	KDS Module	37
3.3	Frontend Structure	38
3.3.1	Front Staff UI - Order Feature	38
3.3.2	Kitchen Staff UI - KDS Feature	40
3.4	Implement Features	42
3.4.1	Order Feature	42
3.4.2	KDS Feature	46
4	Apply SOLID Principles	48
4.1	Architecture-Design Level	48
4.2	Five Principles Applied	49
4.3	Implementation Level	51
4.4	Reflection Report	52

1 Context of the IRMS

1.1 Project Overview

The Intelligent Restaurant Management System (IRMS) is a software application designed to optimize and automate core restaurant operations. The system serves as a centralized digital backbone that coordinates communication between front-of-house staff, kitchen teams, and management through digital interfaces, shared databases, and software-driven logic.

The primary goal of IRMS is to create a scalable, modular, and maintainable architecture that supports dynamic workflows in a restaurant environment. The system manages activities such as menu browsing, order placement, food preparation workflow, table status management, and billing, ensuring smooth interactions between all stakeholders while minimizing delays and errors.

The system adapts to different service conditions, such as peak hours or large group orders, making workflow adjustments accordingly. It also provides management tools for tracking performance metrics, scheduling employees, updating menus, and analyzing sales trends.

1.2 Objectives & Scope

Objectives

- **Primary Goal:** Streamline restaurant operations by enabling efficient order taking, kitchen coordination, and customer service management.
- **Architecture Goals:**
 - Scalable: Handle increasing workloads such as more tables and peak-hour demand.
 - Modular: Divide the system into independent, well-defined modules.
 - Maintainable: Ensure ease of updates, bug fixes, and future extensions.
- **Operational Goals:** Ensure smooth interaction between:
 - Front-of-house staff (servers, hosts, cashiers)
 - Kitchen teams (chefs, prep stations)
 - Management (administrators, managers)
- **Design Goal:** Follow SOLID principles to create a clean, extensible, and well-structured system.

Scope of the System

1. Digital Ordering & Menu Management — Order placement, real-time menu updates, customizations, allergy notes, and combo handling.
2. Kitchen Order Display & Workflow Coordination — Automatic routing to KDS, station-based organization, visual alerts, and progress tracking.
3. Table & Reservation Management — Seating charts, table monitoring, reservations, waitlists, and notifications.
4. Billing, Payments & Receipts — Integrated billing, multi-payment support, split bills, and tip management.
5. Inventory Monitoring — Ingredient tracking, low-stock alerts, and historical usage analysis.
6. Analytics & Reports — Sales reports, operational analytics, and exportable reports.
7. Administrative Tools — Role-based access control, menu configuration, pricing, and audit logging.

1.3 Stakeholders and Actors

The IRMS is designed around the needs of restaurant operations, where different groups interact with the system or are affected by it in different ways.

Stakeholders

Stakeholders are the parties whose interests are affected by the system, even if they do not interact with it directly. The main stakeholders and their concerns are:

- **Restaurant owners and managers** – need accurate sales reports, operational analytics, and centralised control over menu configuration, pricing, and staff records.
- **Front-of-house and kitchen staff** – work in a fast-paced, high-noise environment and need intuitive interfaces that minimise errors and training time.
- **Customers** – do not interact with the system directly, but are affected by service speed, order accuracy, and billing correctness. The system must support staff in delivering a smooth dining experience.
- **System administrators** – responsible for role-based access control, audit logging, and secure handling of critical actions such as refunds, cancellations, and configuration changes.

Actors

Actors are the user groups that directly interact with the IRMS. The main actors are:

- **Front-of-house staff** – servers, hosts, and cashiers who handle table coordination, ordering, billing, and customer service.
- **Kitchen staff** – chefs and prep staff who receive orders through the Kitchen Display System and update preparation status.
- **Managers** – monitor restaurant performance, configure menus and pricing, and review reports.
- **System administrators** – manage user accounts, assign roles, and review audit logs.

These stakeholder and actor definitions are used consistently throughout the functional requirements, architectural views, and system design decisions.

1.4 Functional Requirements

- **Digital Ordering & Menu Management**
 - Staff can take orders via tablets or terminals.
 - Menu items can be updated in real time.
 - Orders support customizations, allergy notes, and special instructions.
 - Menu browsing for both customers and staff.
- **Kitchen Order Display & Workflow Coordination**
 - Orders are automatically routed to the KDS.
 - Orders are organized by preparation time, category, and station.
 - Visual alerts for deadlines and new orders.
 - Kitchen staff can update order status (Started, Cooking, Ready).
- **Table & Reservation Management**
 - Digital seating chart with status tracking.
 - Monitoring of table service and billing status.
 - Reservation scheduling and waitlist management.
- **Billing, Payments & Receipts**
 - Integrated billing system.
 - Support for cash, card, and digital wallet payments.
 - Split bills and tip management.
- **Inventory Monitoring**
 - Manual tracking of ingredient usage.

- Low-stock alerts.
- Historical usage tracking for prediction and waste management.
- **Analytics & Reports**
 - Sales and revenue reports.
 - Operational analytics (delays, efficiency).
 - Exportable reports.
- **Administrative Tools**
 - Role-based access control.
 - Centralized configuration for menu and pricing.
 - Audit logs for critical actions.

1.5 Non-functional Requirements

- **Performance & Responsiveness:** The system must handle peak hours efficiently with low latency and real-time updates.
- **Scalability:** Adapt to varying workloads and service conditions.
- **Maintainability & Modularity:** Support easy updates and modular design.
- **Extensibility:** Allow new features (e.g., payment methods, kitchen stations) without major system changes.
- **Security:** Ensure role-based access control, audit logging, and secure payment processing.
- **Reliability & Availability:** Ensure continuous operation during service hours with no data loss.
- **Usability:** Provide intuitive interfaces for staff operating under high-pressure conditions.

2 Software Architecture for the IRMS

2.1 Architecture Characteristics

- **Performance:** In a restaurant environment, delays in order transmission or updates can lead to service bottlenecks, incorrect orders, and poor customer experience. Orders must be instantly visible to kitchen staff, and updates must propagate quickly across all interfaces.

Key considerations:

- Low latency in order submission and updates
 - Fast synchronization between modules (ordering, kitchen, billing)
 - Efficient handling of concurrent requests during peak hours
 - **Scalability:** Restaurants often experience sudden spikes in demand (e.g., lunch/dinner rush). The system must scale to accommodate increased numbers of users, orders, and transactions. Additionally, future expansion to multiple branches requires scalable design.
- Key considerations:
- Horizontal scaling support
 - Load balancing across services
 - Ability to handle multiple concurrent users and devices
 - **Reliability and Availability:** The IRMS is mission-critical; any system failure can disrupt restaurant operations. Orders must not be lost, and the system must remain operational during business hours.

Key considerations:

- Fault tolerance and error recovery mechanisms
- High system uptime

- Immediate persistence of critical data
- **Usability:** Restaurant staff operate in a fast-paced environment and require intuitive interfaces to minimize errors and training time. Poor usability can lead to incorrect orders and slower service.

Key considerations:

- Simple and intuitive user interfaces
- Minimal steps for common tasks (e.g., order entry)
- Clear visual feedback and alerts
- **Maintainability:** The IRMS is expected to evolve continuously due to changing business requirements such as menu updates, pricing adjustments, workflow improvements, and new feature integrations (e.g., delivery services or loyalty programs). A system that is difficult to maintain will increase development time, introduce bugs, and slow down the deployment of new features.

Key considerations:

- Clear and consistent code structure
- High readability and proper documentation
- Separation of concerns across modules
- Reusability of components
- Ease of debugging and testing

2.2 Architecture Style

In this project, two main architecture styles are considered: **Layered Architecture (Monolithic)** and **Service-Based Architecture**. Each style provides a different way to organize system components and responsibilities.

Layered Architecture (Monolithic)

The **Layered Architecture** organizes the system based on technical responsibilities. Its topology is typically divided into four horizontal layers:

- **Presentation Layer:** handles the user interface and communication with users or external clients.
- **Business Layer:** contains business rules and application logic.
- **Persistence Layer:** manages data access and interaction with storage mechanisms.
- **Database Layer:** stores and maintains the actual application data.

This style is simple, easy to understand, and straightforward to implement, especially for small or less complex systems. It also provides a basic separation of concerns because each layer has a clearly defined technical role.

However, Layered Architecture has several limitations. Since requests usually pass through multiple layers in sequence, some operations may go through unnecessary processing steps even when not all layers are needed. This can lead to the *sinkhole anti-pattern*, where requests move through the architecture without enough meaningful processing to justify the cost. In addition, the layers are often tightly connected in practice, which makes the system harder to scale, test, and deploy independently as it grows.

Service-Based Architecture

The **Service-Based Architecture** organizes the system based on business capabilities or functional domains rather than purely technical layers. In this style, the application is divided into multiple services, where each service is responsible for a specific business area.

This style improves modularity by grouping related responsibilities into domain-oriented components. Each service can evolve more independently and provides a clearer boundary around a specific part of the business. As a result, the architecture is easier to maintain and extend when the system grows in size or complexity.

In this project, the service-based style can also be combined with layered design inside each service. A service may internally include:

- **API Facade Layer:** receives requests and coordinates the service workflow.

- **Business Logic Layer:** implements domain rules and processing logic.
- **Persistence Layer:** communicates with the database or storage system.

This combination allows the architecture to keep domain-based modularity at the system level while still preserving internal structural clarity inside each service.

Comparison of Styles

In summary, the Layered Architecture emphasizes technical separation through horizontal layers, while the Service-Based Architecture emphasizes business separation through domain-oriented services. The layered style is generally simpler and more direct, whereas the service-based style provides stronger modularity and better support for larger and evolving systems.

Feature	Layered Architecture (Monolithic)	Service-Based Architecture
Organization Focus	Technical responsibilities (horizontal layers).	Business capabilities or functional domains.
System Structure	Divided into Presentation, Business, Persistence, and Database layers.	Divided into multiple services (which may internally use API Facade, Business Logic, and Persistence layers).
Key Strengths	Simple, easy to understand, straightforward to implement, and provides a basic technical separation of concerns.	Strong modularity, independent evolution of services, clear business boundaries, and easier to maintain/extend.
Key Weaknesses	Tightly connected (harder to scale/test/deploy independently) and prone to the “sinkhole anti-pattern” (unnecessary processing steps).	<i>(Not explicitly stated in text, but generally implies more complexity than the layered approach).</i>
Best Suited For	Small or less complex systems.	Larger, evolving, or complex systems.

Table 1: Comparison of Architecture Styles

2.3 Architecture Documents

2.3.1 Module View

The Module View captures the static structure of the IRMS, showing how the system is decomposed into packages, classes, and their dependencies. This view is presented through these complementary diagrams: a Package Diagram (high-level service composition), Class Diagrams (per-module class structure).

Package Diagram

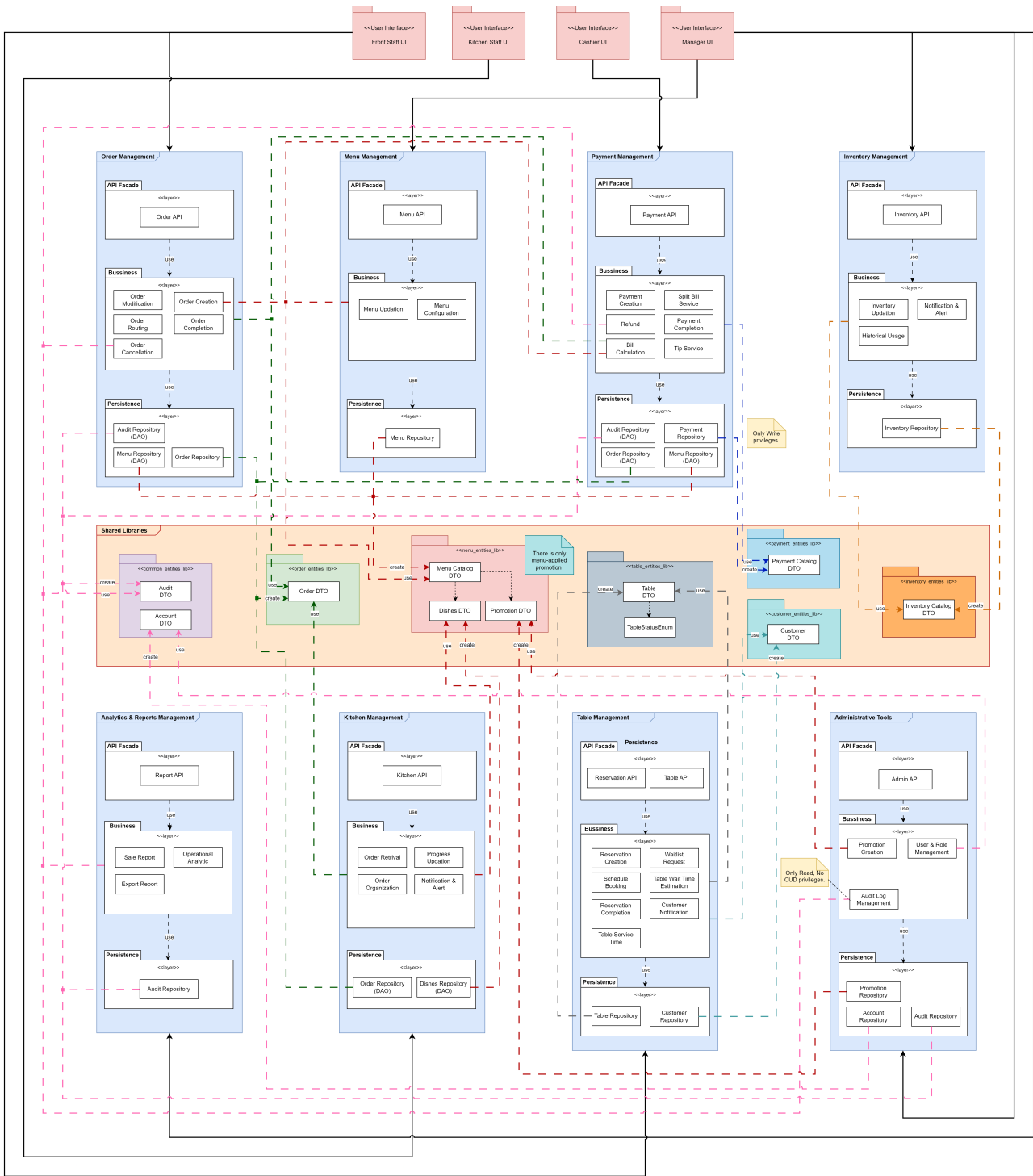


Figure 1: Package Diagram for IRMS system

1. UI Design based on Architecture Quantum

The system is designed using the idea of Architecture Quantum from *Fundamentals of Software Architecture*. We group the system into four role-based UIs: **Front Staff UI**, **Kitchen Staff UI**, **Cashier UI**, and **Manager UI**.

(a) *High Functional Cohesion*

Each UI represents a complete business function for a specific role.

For example, the Front Staff UI includes table management, reservation handling, and order taking. These functions are used together in real work, so grouping them improves cohesion.

This avoids splitting related tasks into separate UIs, which would make the workflow harder.

(b) ***Domain-Based UI Structure***

The design follows the Domain-Based User Interface pattern.

Each UI is organized by business role instead of technical services.

This makes the system easier to understand because each UI matches a real job role in the restaurant.

(c) ***Independent Deployability***

Each UI can be deployed as one independent application.

For example, the Kitchen UI runs on kitchen screens, and the Front Staff UI runs on tablets.

This reduces system complexity and makes deployment and maintenance easier.

(d) ***No Customer UI***

The system is an internal system only.

Customers do not directly use the system.

All customer interactions (like reservation or ordering) are handled by staff.

The system can still send notifications to customers through email or phone when needed.

2. Services

All services are designed using a layered architecture, including three main layers: API Layer, Business Layer, and Persistence Layer.

The API layer receives requests from the UI and handles request routing to the appropriate business functions.

The Business Layer contains the core logic of service, The Business Layer does not directly access the database. Business Layer works with DTOs defined in the shared library. By using DTOs from a **shared library**, it ensures that all services use consistent data structures.

The Persistence Layer is responsible for data access and database interaction. It follows the DAO (Data Access Object) pattern. Each DAO retrieves raw data from the database and maps it into DTOs defined in the shared library.

(a) **Order Management**

- ***API Layer***

The API Layer exposes the *Order API* as the entry point for all order-related operations.

This layer is responsible for:

- Receiving HTTP requests
- Basic input validation
- Mapping requests to business services

- ***Business Layer***

It implements all order-related workflows, including:

- Order creation
- Order modification
- Order routing to kitchen or cashier
- Order completion
- Order cancellation

Key usage of DTOs:

- *Order DTO*: used for creating, updating, and processing orders
- *Menu DTO*: used to validate and retrieve menu item information
- *Audit DTO*: used during order cancellation to store logs and tracking information

- ***Persistence Layer***

This layer includes:

- *Order Repository (DAO)*: handles order data storage and retrieval
- *Menu Repository (DAO)*: provides menu item data
- *Audit Repository (DAO)*: stores audit logs for tracking and debugging

(b) **Menu Management**

- ***API Layer***

The API Layer exposes the *Menu API* as the entry point for all menu-related requests.

This layer is responsible for:

- Receiving HTTP requests for menu operations
- Basic validation of input data
- Routing requests to business services
- **Business Layer**
It handles all operations related to menu items and configuration.
Main functions include:
 - Menu creation and updating
 - Menu item modification (price, name, availability)
 - Menu configuration management (e.g., categories, status)
 It uses *Menu DTO* from the shared library to ensure consistent data structures across the system.
- **Persistence Layer**
It includes:
 - *Menu Repository (DAO)*: handles all menu-related database operations

(c) Payment Management

- **API Layer**
The API Layer exposes the *Payment API* as the entry point for all payment-related operations.
This layer is responsible for:
 - Receiving HTTP requests
 - Basic input validation
 - Mapping requests to business services
- **Business Layer**
It implements all payment-related workflows, including:
 - Payment creation: generate bill from order data and prepare payment request
 - Payment completion: finalize successful transactions
 - Split bill handling: divide payment between multiple customers
 - Refund processing: handle refund requests when needed
 - Tip calculation: compute and distribute service tips
 Key usage of DTOs:
 - *Payment DTO*: used for payment creation and processing transactions
 - *Order DTO*: used to retrieve order information for billing
 - *Menu DTO*: used to access item price and menu details
 - *Audit DTO*: used to store payment logs, refund history, and traceability data
- **Persistence Layer**
This layer includes:
 - *Payment Repository (DAO)*: handles payment transaction storage and retrieval
 - *Order Repository (DAO)*: provides order data for billing purposes
 - *Menu Repository (DAO)*: provides menu item pricing and details
 - *Audit Repository (DAO)*: stores logs for payment tracking, refund, and auditing

(d) Inventory Management

- **API Layer**
The API Layer exposes the *Inventory API* as the entry point for all inventory-related operations.
This layer is responsible for:
 - Receiving HTTP requests for inventory updates and queries
 - Basic input validation
 - Mapping requests to business services
- **Business Layer**
It implements all inventory-related workflows, including:
 - Inventory updating: update stock when items are used or restocked
 - Inventory notification and alerts: notify staff when stock is low or out of stock
 - Historical usage tracking: record and analyze inventory usage over time
 Key usage of DTOs:
 - *Inventory DTO*: used for managing stock information and updates

- **Persistence Layer**

This layer includes:

- *Inventory Repository (DAO)*: handles storage and retrieval of inventory data

(e) **Analytics & Reports Management**

- **API Layer**

The API Layer exposes the *Analytics API* as the entry point for all reporting and analytics-related requests.

This layer is responsible for:

- Receiving report and analytics queries
- Validating parameters (e.g., date range, report type)
- Routing requests to analytics services

- **Business Layer**

It processes and aggregates data retrieved from multiple services through the shared database.

Main functions include:

- Sales analytics: show revenue performance, peak hours, best-selling items
- Operational analytics: highlight delays, kitchen bottlenecks, and staff efficiency
- Export Report: export the reports for accounting, planning, or performance reviews.

Key usage of DTOs:

- *Audit DTO* for system activity tracking

- **Persistence Layer**

This layer retrieves data from the **shared database** via DAO components.

It includes:

- *Audit Repository (DAO)*: store logs for conduct Sale report and operational analytics

(f) **Kitchen Management**

- **API Layer**

The API Layer exposes the *Kitchen API* as the entry point for all kitchen-related operations.

This layer is responsible for:

- Receiving order data routed from front-of-house systems
- Providing endpoints for kitchen staff to update order progress
- Handling real-time requests from the Kitchen Display System (KDS)

- **Business Layer**

The Business Layer is structured into specialized components that reflect the kitchen workflow:

- Order Retrieval: retrieves incoming orders from the system and prepares them for kitchen processing
- Order Organization: organizes orders based on preparation time, dish category, and kitchen station (e.g., grill, fryer, dessert), supporting efficient workflow coordination in the KDS
- Progress Updation: manages order status updates such as *QUEUED*, *PREPARING*, and *READY*, ensuring real-time synchronization with other services
- Notification & Alert: generates visual alerts for new orders and warnings for orders nearing preparation deadlines

These components directly support the Kitchen Order Display and Workflow Coordination requirements by ensuring orders are properly routed, prioritized, and tracked.

Key usage of DTOs:

- *Order DTO*: used to retrieve and update order status and workflow information
- *Dishes DTO*: used to determine dish category and preparation requirements for organization and routing

- **Persistence Layer**

The Persistence Layer handles data access through DAO components.

It includes:

- *Order Repository (DAO)*: retrieves order data and updates order progress status
- *Dishes Repository (DAO)*: provides dish information such as category and preparation details

(g) **Table Management**

- **API Layer**

The API Layer exposes two entry points: the *Table API* and the *Reservation API*.

This layer is responsible for:

- Receiving requests from the Table/Reservation UI used by hosts and servers
- Handling table status updates and reservation operations
- Triggering customer notification processes through external communication channels (e.g., email, phone)

- **Business Layer**

The Business Layer is decomposed into components that directly support table and reservation workflows:

- Reservation Creation: handles creation of new reservations based on customer requests received by staff
- Schedule Booking: manages reservation scheduling, including date, time, and table allocation
- Reservation Completion: finalizes reservations when customers arrive or when bookings are fulfilled
- Waitlist Request: manages walk-in customers and maintains a queue when no tables are immediately available
- Table Wait Time Estimation: calculates estimated waiting time based on current table occupancy and service duration
- Table Service Time: tracks how long a table has been occupied to support operational monitoring
- Customer Notification: sends automated notifications (e.g., booking confirmation, waitlist updates) to customers via external systems

This design reflects the system constraint that all interactions are handled internally by staff. Customers do not directly interact with the system; instead, hosts input reservation and waitlist data, and the system communicates outward via notification mechanisms.

Key usage of DTOs:

- *Table DTO*: used for managing table status and availability.
- *Customer DTO*: used for customer notification and managing customer information.

- **Persistence Layer**

The Persistence Layer manages data access through DAO components.

It includes:

- *Table Repository (DAO)*: stores and retrieves table status, reservations, and service time data
- *Customer Repository (DAO)*: stores customer contact information used for reservations and notifications

(h) Administrative Tools

- **API Layer**

The API Layer consists of the *Admin API*.

- Admin API: provides specific endpoints for administrative functions such as user management, promotion configuration, and audit retrieval

This design reduces complexity for the client side and ensures consistent access control enforcement across all administrative operations.

- **Business Layer**

The Business Layer is organized into core administrative components:

- User & Role Management: implements role-based access control (RBAC), managing user accounts and permissions for managers, servers, chefs, and cashiers
- Promotion Creation: handles creation and management of promotional campaigns, including pricing adjustments and special offers applied to menu items
- Audit Log Management: records and manages logs of critical system actions such as refunds, order cancellations, and configuration changes

These components support centralized system administration, ensuring consistent configuration, controlled access, and traceability of important operations.

Key usage of DTOs:

- *Account DTO*: used for managing user accounts and roles
- *Audit DTO*: used for logging and tracking administrative actions
- *Menu DTO*: used when applying promotions and price updates

- **Persistence Layer**

The Persistence Layer manages data access through DAO components.

It includes:

- *Account Repository (DAO)*: stores and retrieves user account and role data
- *Promotion Repository (DAO)*: manages promotional campaign data
- *Audit Repository (DAO)*: stores audit logs for system traceability

3. Shared Libraries

The Shared Libraries contain all Data Transfer Objects (DTOs) and common data models used across the system. These libraries ensure consistency of data representation between all services and reduce duplication of model definitions.

Each service (Order, Menu, Payment, Inventory, Table, etc.) uses these shared DTOs to exchange data in a consistent format.

- ***Common Entities Library***

This library contains shared system-wide DTOs used by multiple services.

- *Audit DTO*: used for logging, tracking, and system auditing
- *Account DTO*: used for user account and authentication-related data

- ***Order Entities Library***

This library defines data models related to order processing.

- *Order DTO*: represents order information used across services

- ***Menu Entities Library***

This library defines menu-related data structures.

- *Menu Catalog DTO*: represents menu catalog information
- *Dishes DTO*: represents individual dish data
- *Promotion DTO*: represents menu-based promotions (only applied to menu items)

- ***Table Entities Library***

This library defines table management data models.

- *Table DTO*: represents table information
- *TableStatusEnum*: defines table states (e.g., available, occupied, reserved)

- ***Customer Entities Library***

This library defines customer-related data models used for reservation and notification workflows.

- *Customer DTO*: represents customer information such as name, contact details (phone number, email), and reservation history

- ***Payment Entities Library***

This library defines payment-related data models.

- *Payment Catalog DTO*: represents payment transaction data and billing information

- ***Inventory Entities Library***

This library defines inventory-related data models.

- *Inventory Catalog DTO*: represents stock items and inventory status

The Shared Libraries act as a common contract between services, ensuring all modules communicate using the same structured data format.

Class Diagram

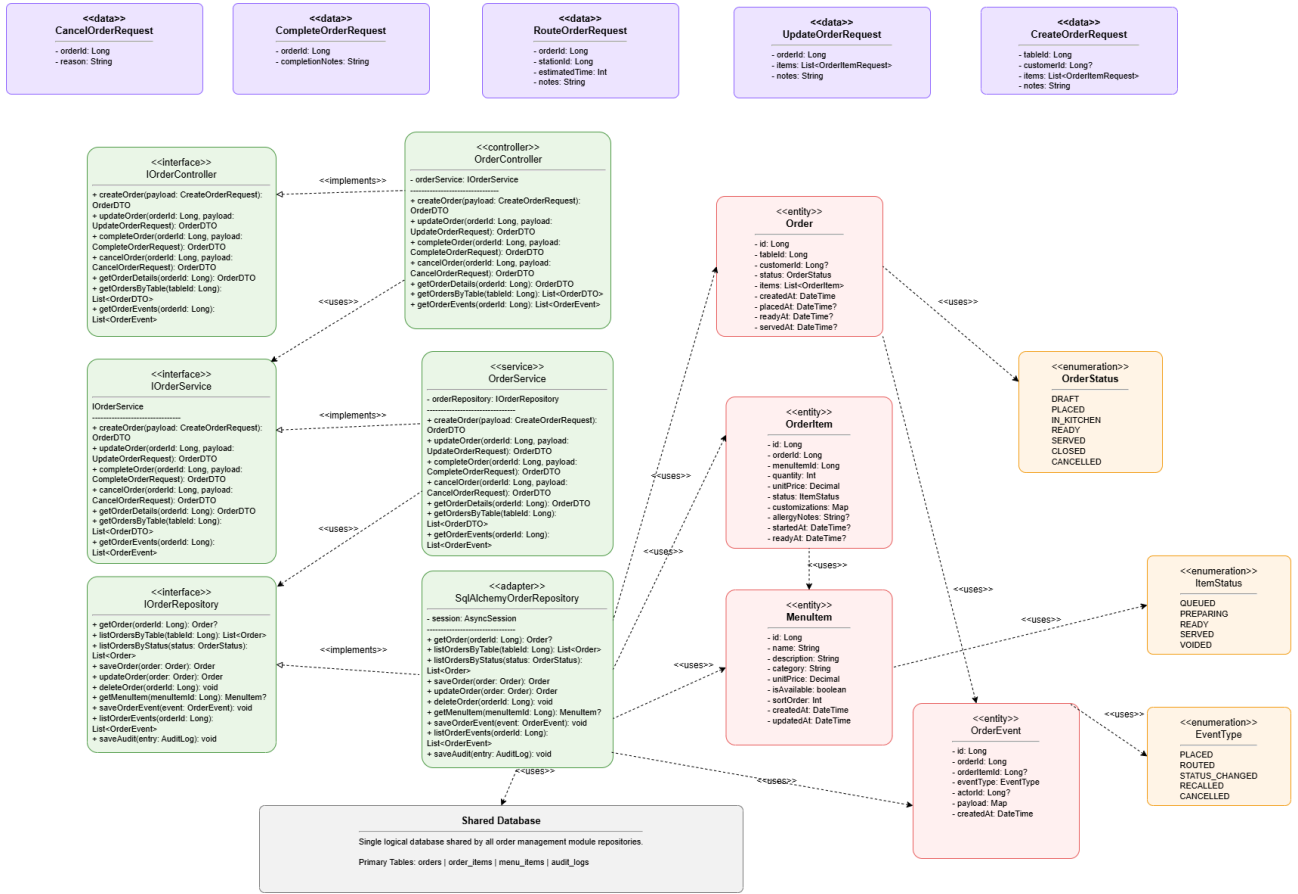


Figure 2: Class diagram for Order Module.

Order Module The Order Module manages the restaurant order lifecycle, including order creation, item updates, completion, cancellation, and event tracking. It follows the layered structure of Controller, Business, and Persistence layers. Each implementation depends on an abstract interface, using `<<implements>>` for realization and `<<uses>>` for dependencies.

Controller Layer.

- **IOrderController** (*interface*) – defines the external controller contract for order operations, including `createOrder`, `updateOrder`, `completeOrder`, `cancelOrder`, `getOrderDetails`, `getOrdersByTable`, and `getOrderEvents`.
- **OrderController** (*controller*) – implements **IOrderController**; receives order-related requests from staff-facing UIs, validates request payloads, and delegates business logic to **IOrderService**. It exposes operations for creating, updating, completing, cancelling, querying orders, and retrieving order events.

Business Layer.

- **IOrderService** (*interface*) – defines the business contract for the order workflow. It includes order creation, modification, completion, cancellation, detail lookup, table-based listing, and order event retrieval.
- **OrderService** (*service*) – implements **IOrderService**; coordinates the order lifecycle and applies business rules before delegating persistence operations to **IOrderRepository**. It manages order creation, order item updates, order completion, cancellation, and event logging through **OrderEvent**.

Persistence Layer.

- **IOrderRepository** (*interface*) – defines the persistence contract for order-related data. Main operations include `getOrder`, `listOrdersByTable`, `listOrdersByStatus`, `saveOrder`, `updateOrder`, `deleteOrder`, `getMenuItem`, `saveOrderEvent`, `listOrderEvents`, and `saveAudit`.
- **SqlAlchemyOrderRepository** (*adapter*) – implements **IOrderRepository**; backed by an asynchronous SQLAlchemy session (`AsyncSession`). It is responsible for reading and writing **Order**, **OrderItem**, **MenuItem**, **OrderEvent**, and audit-related data in the shared database.

Domain Entities.

- **Order** (*persisted*) – represents a restaurant order. Fields: `id: Long`, `tableId: Long`, `customerId: Long?`, `status: OrderStatus`, `items: List<OrderItem>`, `createdAt: DateTime`, `placedAt: DateTime?`, `readyAt: DateTime?`, and `servedAt: DateTime?`. It uses `OrderStatus` to describe the lifecycle state of the order.
- **OrderItem** (*persisted*) – represents an item inside an order. Fields: `id: Long`, `orderId: Long`, `menuItemId: Long`, `quantity: Int`, `unitPrice: Decimal`, `status: ItemStatus`, `customizations: Map`, `allergyNotes: String?`, `startedAt: DateTime?`, and `readyAt: DateTime?`. It uses `ItemStatus` to track item-level preparation progress.
- **MenuItem** (*read-only reference*) – represents menu information required by the order workflow. Fields: `id: Long`, `name: String`, `description: String`, `category: String`, `unitPrice: Decimal`, `isAvailable: Boolean`, `sortOrder: Int`, `createdAt: DateTime`, and `updatedAt: DateTime`. The Order Module uses this entity to validate ordered items and retrieve item pricing.
- **OrderEvent** (*persisted*) – records important lifecycle events of an order. Fields: `id: Long`, `orderId: Long`, `orderItemId: Long?`, `eventType: EventType`, `actorId: Long?`, `payload: Map`, and `createdAt: DateTime`. It provides an audit trail for actions such as placing, routing, status changes, recalls, and cancellations.

Enumerations.

- **OrderStatus** – defines order-level lifecycle states: `DRAFT`, `PLACED`, `IN_KITCHEN`, `READY`, `SERVED`, `CLOSED`, and `CANCELLED`.
- **ItemStatus** – defines item-level preparation states: `QUEUED`, `PREPARING`, `READY`, `SERVED`, and `VOIDED`.
- **EventType** – defines order event categories: `PLACED`, `ROUTED`, `STATUS_CHANGED`, `RECALLED`, and `CANCELLED`.

Data Transfer Objects.

- **CreateOrderRequest** – payload for creating an order. Fields: `tableId: Long`, `customerId: Long?`, `items: List<OrderItemRequest>`, and `notes: String`.
- **UpdateOrderRequest** – payload for updating an existing order. Fields: `orderId: Long`, `items: List<OrderItemRequest>`, and `notes: String`.
- **CompleteOrderRequest** – payload for completing an order. Fields: `orderId: Long` and `completionNotes: String`.
- **CancelOrderRequest** – payload for cancelling an order. Fields: `orderId: Long` and `reason: String`.
- **RouteOrderRequest** – payload for routing an order to preparation. Fields: `orderId: Long`, `stationId: Long`, `estimatedTime: Int`, and `notes: String`.
- **OrderDTO** – response object returned by order operations. It represents order details together with item information and current lifecycle status.

IRMS - Class Diagram for Menu Module

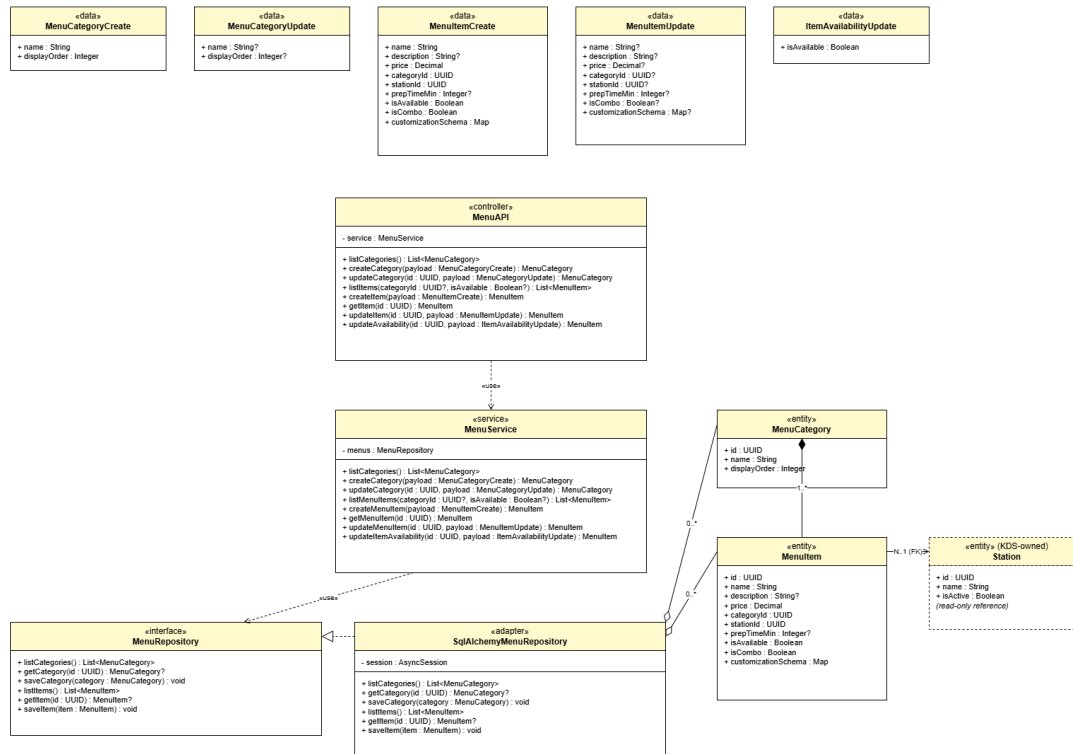


Figure 3: Class diagram for Menu Module.

Menu Module The Menu Module manages the menu catalogue, including categories and items. It follows the layered structure of API, Business, and Persistence layers, with each layer accessing the next through an abstract interface. **Station** is referenced read-only via foreign key but owned by Kitchen Display.

API Layer.

- **MenuAPI** (*controller*) – receives HTTP requests, validates payloads, and delegates business logic to **MenuService**. Exposes the REST endpoints: `listCategories`, `createCategory`, `updateCategory`, `listItems`, `createItem`, `getItem`, `updateItem`, `updateAvailability`.

Business Layer.

- **MenuService** (*service*) – orchestrates category and item lifecycle (creation, modification, listing, availability updates) and delegates persistence to **MenuRepository**. Method signatures use the explicit **MenuItem** prefix (e.g., `createMenuItem`) to distinguish from REST-facing names. Exposes a dedicated `updateItemAvailability` operation to support high-frequency availability toggling without requiring the full item payload.

Persistence Layer.

- **MenuRepository** (*interface*) – defines the persistence contract: `saveCategory`, `saveItem`, `getCategory`, `getItem`, `listCategories`, `listItems`. Decouples the business layer from any specific storage technology, in line with the Dependency Inversion Principle.
- **SQLAlchemyMenuRepository** (*adapter*) – implements **MenuRepository**; backed by an asynchronous SQLAlchemy session (**AsyncSession**).

Domain Entities.

- **MenuCategory** (*persisted*) – displayable grouping for menu items. Fields: `id`: UUID (primary key), `name`: String, `displayOrder`: Integer (used to sort categories on the customer-facing UI). Aggregates **MenuItem** (1..*).
- **MenuItem** (*persisted*) – core menu catalogue entry. Fields: `id`: UUID (primary key), `name`: String, `description`: String?, `price`: Decimal, `categoryId`: UUID, `stationId`: UUID, `prepTimeMin`: Integer?, `isAvailable`: Boolean, `isCombo`: Boolean, `customizationSchema`: Map (flexible per-item customisation options such as size or toppings). References **MenuCategory** and **Station** (N..1 FK).

- **Station** (*KDS-owned, read-only reference*) – referenced via `stationId` FK; owned by the Kitchen Display Module (see Figure 6). Fields: `id`: `UUID`, `name`: `String`, `isActive`: `Boolean`. The Menu Module never writes to this entity; it only stores `stationId` on `MenuItem` so that orders can later be routed to the correct kitchen station.

Data Transfer Objects.

- **MenuCategoryCreate** – payload for category creation (`name`: `String`, `displayOrder`: `Integer`).
- **MenuCategoryUpdate** – payload for partial category update; all fields optional, supporting PATCH semantics.
- **MenuItemCreate** – payload for item creation with all required fields including pricing (`price`), kitchen routing (`stationId`), and customisation schema (`customizationSchema`).
- **MenuItemUpdate** – payload for partial item update; all fields optional, supporting PATCH semantics.
- **ItemAvailabilityUpdate** – minimal payload dedicated to toggling `isAvailable` without exposing other fields, used by the dedicated availability endpoint for fast operations such as marking an item as sold out.

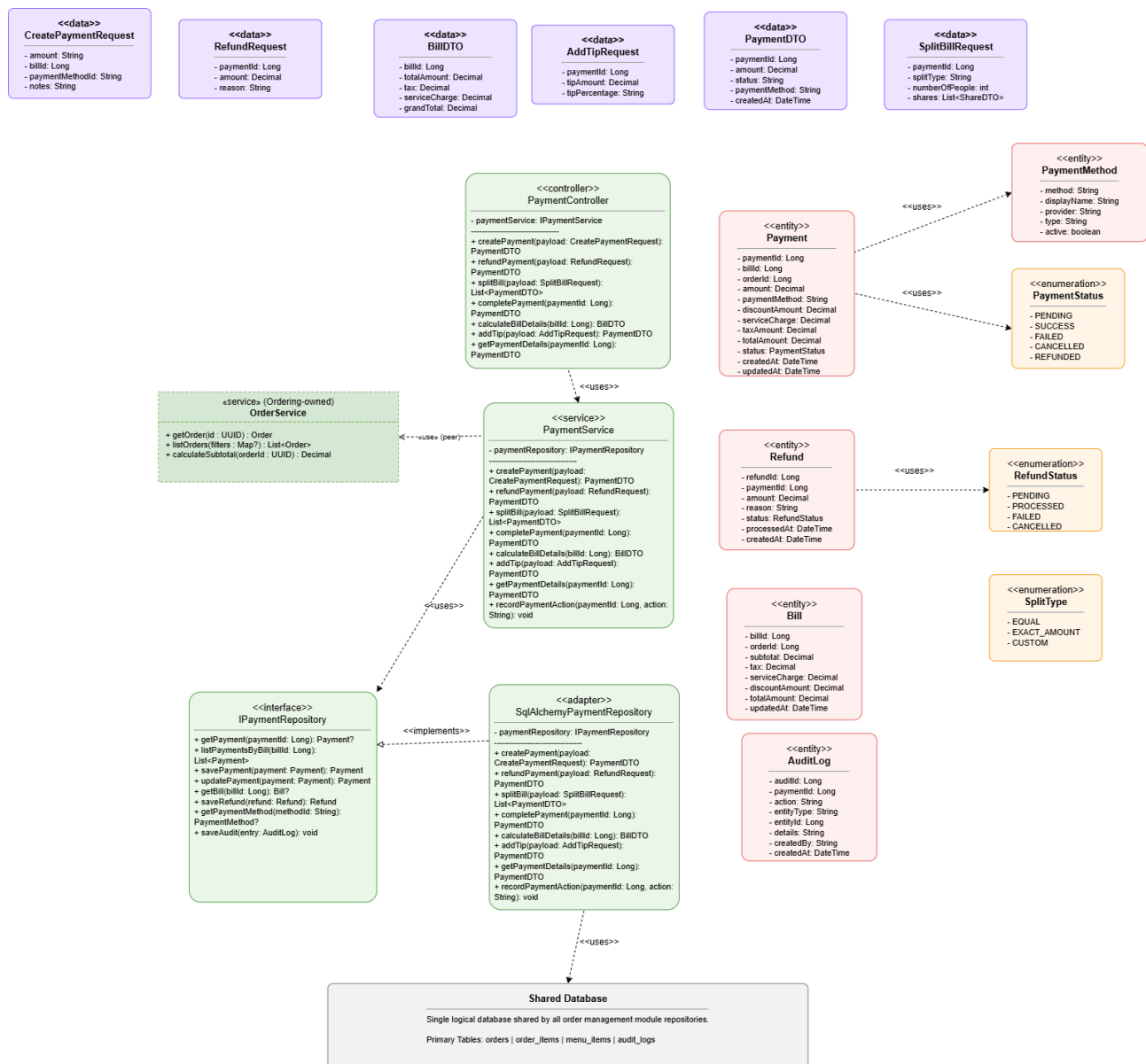


Figure 4: Class diagram for Payment Module.

Payment Module The Payment Module manages billing and payment-related operations, including payment creation, refund handling, split-bill processing, tip calculation, payment completion, and payment audit

logging. It follows the layered structure of Controller, Business, and Persistence layers. The module also references the Ordering Module through `OrderService` to retrieve order information and calculate order subtotals.

Controller Layer.

- `PaymentController` (*controller*) – receives payment-related requests from the cashier or authorized staff UI, performs basic request validation, and delegates business logic to `IPaymentService`. Exposes operations such as `createPayment`, `refundPayment`, `splitBill`, `completePayment`, `calculateBillDetails`, `addTip`, and `getPaymentDetails`.

Business Layer.

- `PaymentService` (*service*) – orchestrates the payment workflow and implements payment-related business rules. It handles payment creation, refund processing, split-bill calculation, payment completion, bill calculation, tip updates, and audit recording. The service depends on `IPaymentRepository` for persistence operations and uses the ordering-owned `OrderService` as a peer-domain reference for retrieving order data and calculating order subtotals.
- `OrderService` (*Ordering-owned peer service*) – referenced by the Payment Module for read-only order information. It provides operations such as `getOrder`, `listOrders`, and `calculateSubtotal`. This service remains owned by the Ordering Module, so the Payment Module does not directly manage order lifecycle data.

Persistence Layer.

- `IPaymentRepository` (*interface*) – defines the persistence contract for payment-related data. Main operations include `getPayment`, `listPaymentsByBill`, `savePayment`, `updatePayment`, `getBill`, `saveRefund`, `getPaymentMethod`, and `saveAudit`. This interface decouples the business layer from concrete database implementation.
- `SqlAlchemyPaymentRepository` (*adapter*) – implements `IPaymentRepository`; backed by the shared database through a SQLAlchemy-based persistence mechanism. It persists and retrieves payment records, bills, refunds, payment methods, and audit logs.

Domain Entities.

- `Payment` (*persisted*) – represents a payment transaction. Fields: `paymentId`: Long, `billId`: Long, `orderId`: Long, `amount`: Decimal, `paymentMethod`: String, `discountAmount`: Decimal, `serviceCharge`: Decimal, `taxAmount`: Decimal, `totalAmount`: Decimal, `status`: `PaymentStatus`, `createdAt`: DateTime, and `updatedAt`: DateTime. It uses `PaymentStatus` to describe the payment lifecycle state and references `PaymentMethod` for the selected payment method.
- `Bill` (*persisted*) – represents the calculated bill for an order. Fields: `billId`: Long, `orderId`: Long, `subtotal`: Decimal, `tax`: Decimal, `serviceCharge`: Decimal, `discountAmount`: Decimal, `totalAmount`: Decimal, and `updatedAt`: DateTime. It is used by the payment workflow to determine the final payable amount.
- `Refund` (*persisted*) – represents a refund transaction linked to an existing payment. Fields: `refundId`: Long, `paymentId`: Long, `amount`: Decimal, `reason`: String, `status`: `RefundStatus`, `processedAt`: DateTime, and `createdAt`: DateTime. It uses `RefundStatus` to track refund processing state.
- `PaymentMethod` (*persisted/reference*) – represents available payment methods. Fields: `method`: String, `displayName`: String, `provider`: String, `type`: String, and `active`: Boolean. It allows the system to distinguish between supported payment options such as cash, card, or other provider-based methods.
- `AuditLog` (*persisted*) – records payment-related actions for traceability. Fields: `auditId`: Long, `paymentId`: Long, `action`: String, `entityType`: String, `entityId`: Long, `details`: String, `createdBy`: String, and `createdAt`: DateTime. It is used to record important events such as payment creation, refund action, split-bill action, and payment completion.

Enumerations.

- `PaymentStatus` – defines payment lifecycle states: PENDING, SUCCESS, FAILED, CANCELLED, and REFUNDED.
- `RefundStatus` – defines refund processing states: PENDING, PROCESSED, FAILED, and CANCELLED.
- `SplitType` – defines split-bill strategies: EQUAL, EXACT_AMOUNT, and CUSTOM.

Data Transfer Objects.

- **CreatePaymentRequest** – payload for creating a payment. Fields: `amount: String`, `billId: Long`, `paymentMethodId: String`, and `notes: String`.
- **RefundRequest** – payload for requesting a refund. Fields: `paymentId: Long`, `amount: Decimal`, and `reason: String`.
- **SplitBillRequest** – payload for splitting a payment. Fields: `paymentId: Long`, `splitType: String`, `numberOfPeople: int`, and `shares: List<ShareDTO>`.
- **AddTipRequest** – payload for adding a tip to a payment or bill. Fields: `paymentId: Long`, `tipAmount: Decimal`, and `tipPercentage: String`.
- **PaymentDTO** – response object returned by payment operations. Fields: `paymentId: Long`, `amount: Decimal`, `status: String`, `paymentMethod: String`, and `createdAt: DateTime`.
- **BillDTO** – response object for bill calculation results. Fields: `billId: Long`, `totalAmount: Decimal`, `tax: Decimal`, `serviceCharge: Decimal`, and `grandTotal: Decimal`.

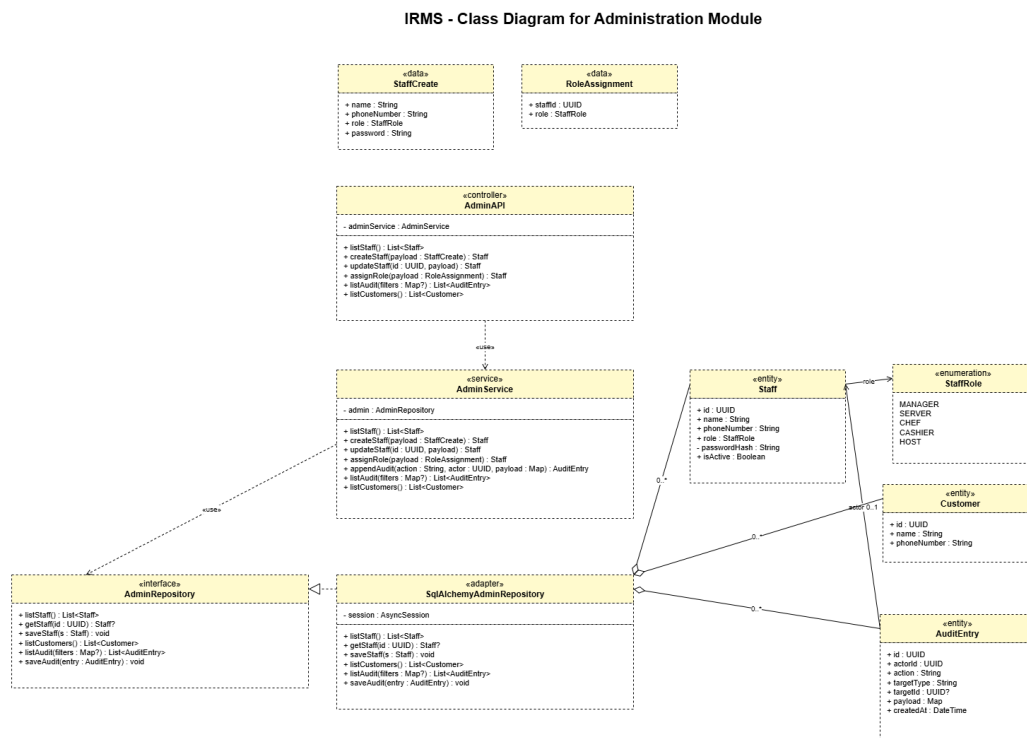


Figure 5: Class diagram for Administration Module.

Administration Module The Administration Module manages staff accounts, role assignments, customer records, and the system-wide audit trail. It follows the layered structure of API, Business, and Persistence layers, with each layer accessing the next through an abstract interface. The module is the authoritative owner of the **Staff**, **Customer**, and **AuditEntry** entities, and exposes the **StaffRole** enumeration consumed by other modules for authorisation.

API Layer.

- **AdminAPI** (*controller*) – receives HTTP requests, validates payloads, and delegates business logic to **AdminService**. Exposes the REST endpoints: `listStaff`, `createStaff`, `updateStaff`, `assignRole`, `listAudit`, `listCustomers`.

Business Layer.

- **AdminService** (*service*) – orchestrates staff lifecycle (creation, modification, role assignment), customer listing, and audit operations, and delegates persistence to **AdminRepository**. Exposes an `appendAudit` operation that records administrative actions as **AuditEntry** records, providing a single point through which auditable events enter the system.

Persistence Layer.

- **AdminRepository** (*interface*) – defines the persistence contract: `listStaff`, `getStaff`, `saveStaff`, `listCustomers`, `listAudit`, `saveAudit`. Decouples the business layer from any specific storage technology, in line with the Dependency Inversion Principle.
- **SqlAlchemyAdminRepository** (*adapter*) – implements **AdminRepository**; backed by an asynchronous SQLAlchemy session (**AsyncSession**). Manages the lifecycle of **Staff**, **Customer**, and **AuditEntry** entities through a *composition* relationship (0..* for each), reflecting that the repository is the authoritative owner of persisted administrative records.

Domain Entities.

- **Staff** (*persisted*) – represents an employee account. Fields: `id`: **UUID** (primary key), `name`: **String**, `phoneNumber`: **String**, `role`: **StaffRole**, `passwordHash`: **String**, `isActive`: **Boolean**. The `passwordHash` field stores a hashed credential rather than the plaintext password supplied at creation time, and the `isActive` flag supports soft deactivation in order to preserve referential integrity with historical **AuditEntry** records. References **StaffRole** through the `role` association.
- **Customer** (*persisted*) – represents a customer record used for order attribution and contact. Fields: `id`: **UUID** (primary key), `name`: **String**, `phoneNumber`: **String**.
- **AuditEntry** (*persisted*) – immutable record of a system action, used for traceability and compliance. Fields: `id`: **UUID** (primary key), `actorId`: **UUID** (foreign key to **Staff**, with 0..1 multiplicity to support system-initiated actions without a human actor), `action`: **String** (action verb such as **STAFF_CREATED** or **ROLE_ASSIGNED**), `targetType`: **String**, `targetId`: **UUID?**, `payload`: **Map** (action-specific contextual data), `createdAt`: **DateTime**.

Enumerations.

- **StaffRole** (*enumeration*) – enumerates the staff roles used for authorisation across the system: **MANAGER**, **SERVER**, **CHEF**, **CASHIER**, **HOST**.

Data Transfer Objects.

- **StaffCreate** – payload for staff creation (`name`: **String**, `phoneNumber`: **String**, `role`: **StaffRole**, `password`: **String**). The plaintext `password` is hashed by the service before persistence; only `passwordHash` is stored on the **Staff** entity.
- **RoleAssignment** – minimal payload dedicated to role changes (`staffId`: **UUID**, `role`: **StaffRole**), used by the `assignRole` endpoint to keep role transitions narrow and auditable independently of other staff updates.

IRMS - Class Diagram for Kitchen Display (KDS) Module

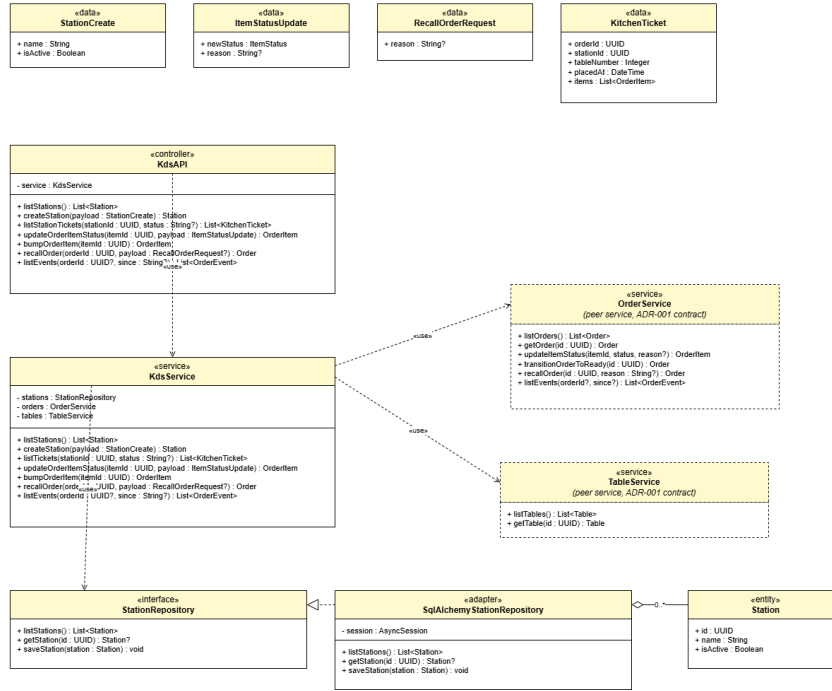


Figure 6: Class diagram for Kitchen Display (KDS) Module.

Kitchen Display (KDS) Module The Kitchen Display Module manages kitchen stations and presents the kitchen-facing view of in-progress orders.

API Layer.

- **KdsAPI** (*controller*) – receives HTTP requests, validates payloads, and delegates business logic to **KdsService**. Exposes the REST endpoints: `listStations`, `createStation`, `listStationTickets`, `updateOrderItemStatus`, `bumpOrderItem`, `recallOrder`, `listEvents`.

Business Layer.

- **KdsService** (*service*) – orchestrates station management and the kitchen-side workflow (ticket listing, item status transitions, bumping, order recall, event streaming). It owns its own data through **StationRepository**, and consults the peer services **OrderService** and **TableService** to assemble **KitchenTicket** projections without holding a copy of order or table state.

Peer Services (External Contracts).

- **OrderService** (*peer service contract*) – exposed by the Order Module and consumed here through a clear service boundary; provides order retrieval, item status transitions, ready-state transitions, recall, and event listing. Drawn with a dashed border to indicate that it is referenced rather than implemented within the KDS Module.
- **TableService** (*peer service contract*) – exposed by the Table Module and consumed here for table lookup when composing kitchen tickets. As with **OrderService**, it is a referenced contract, not an implementation belonging to this module.

Persistence Layer.

- **StationRepository** (*interface*) – defines the persistence contract: `listStations`, `getStation`, `saveStation`. Decouples the business layer from any specific storage technology, in line with the Dependency Inversion Principle.
- **SqlAlchemyStationRepository** (*adapter*) – implements **StationRepository**; backed by an asynchronous SQLAlchemy session (**AsyncSession**). Manages the lifecycle of **Station** entities through a *composition* relationship (0..*), reflecting that the repository is the authoritative owner of persisted station records.

Domain Entities.

- **Station** (*persisted*) – represents a kitchen preparation station (e.g. grill, salad, bar) to which menu items are routed for preparation. Fields: `id`: `UUID` (primary key), `name`: `String`, `isActive`: `Boolean` (used for soft deactivation so that historical `MenuItem.stationId` references remain valid).

Data Transfer Objects.

- **StationCreate** – payload for station creation (`name`: `String`, `isActive`: `Boolean`).
- **ItemStatusUpdate** – payload for transitioning the kitchen-side status of an order item (`newStatus`: `ItemStatus`, `reason`: `String?`). Used by `updateOrderItemStatus` and `bumpOrderItem`.
- **RecallOrderRequest** – minimal payload for the order recall workflow (`reason`: `String?`), used when a completed order must be returned to the kitchen queue.
- **KitchenTicket** – read-only projection assembled by `KdsService` for display on the kitchen screen. Fields: `orderId`: `UUID`, `stationId`: `UUID`, `tableNumber`: `Integer`, `placedAt`: `DateTime`, `items`: `List<OrderItem>`. Aggregates data sourced from `OrderService`, `TableService`, and the local `StationRepository`, and is never persisted.

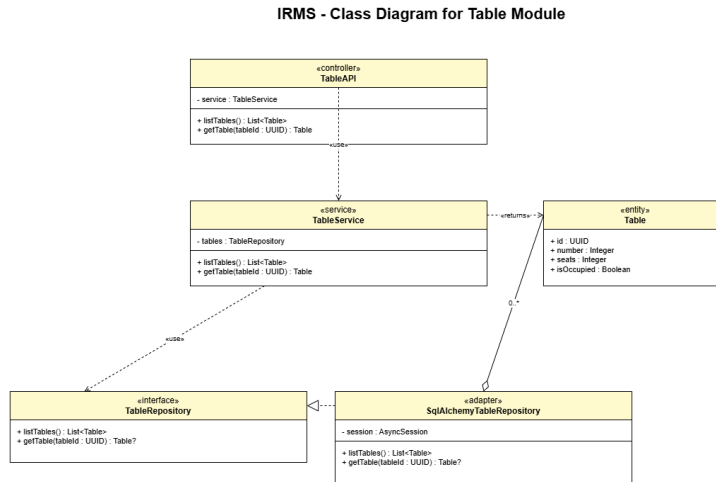


Figure 7: Class diagram for Table Module.

Table Module The Table Module manages the restaurant’s physical table inventory and exposes read-only queries that allow other modules and front-of-house staff to retrieve table information. In the current scope, the module surfaces only listing and lookup operations; occupancy state transitions are driven by the Order Module, which writes the `isOccupied` flag when a dine-in order is opened or closed. This separation keeps the Table Module focused on a single, well-defined responsibility.

API Layer.

- **TableAPI** (*controller*) – receives HTTP requests, validates path parameters, and delegates business logic to **TableService**. Exposes the REST endpoints: `listTables`, `getTable`.

Business Layer.

- **TableService** (*service*) – orchestrates table queries and delegates persistence to **TableRepository**. It is the sole entry point through which both the local controller and other modules access table data; in particular, the Kitchen Display Module consumes **TableService** as a peer service through a well-defined module boundary (see Figure 6).

Persistence Layer.

- **TableRepository** (*interface*) – defines the persistence contract: `listTables`, `getTable`. Decouples the business layer from any specific storage technology, in line with the Dependency Inversion Principle.
- **SQLAlchemyTableRepository** (*adapter*) – implements **TableRepository**; backed by an asynchronous SQLAlchemy session (`AsyncSession`). Manages the lifecycle of **Table** entities through a *composition* relationship (`0..*`), reflecting that the repository is the authoritative owner of persisted table records.

Domain Entities.

- **Table** (*persisted*) – represents a physical dining table. Fields: `id`: `UUID` (primary key), `number`: `Integer` (human-readable table number used by staff), `seats`: `Integer` (seating capacity), `isOccupied`: `Boolean` (current occupancy flag, updated by the Order Module when a dine-in order is opened or closed).

IRMS - Class Diagram for Inventory Module

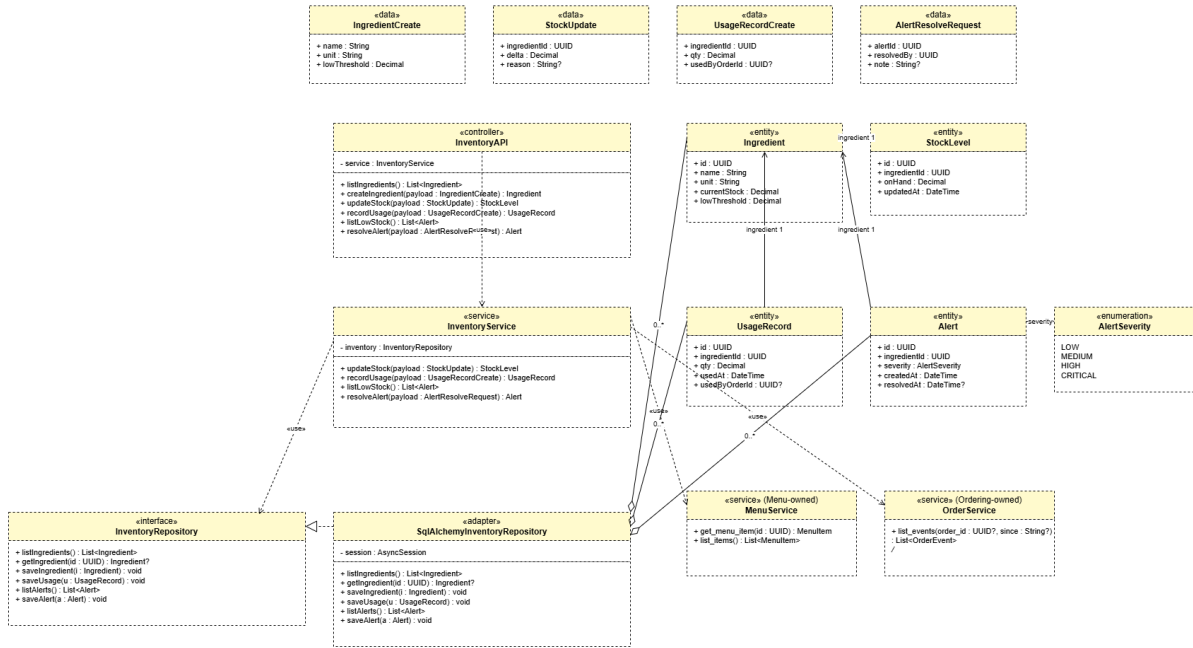


Figure 8: Class diagram for Inventory Module.

Inventory Module The Inventory Module manages the stock of raw ingredients, records consumption against served orders, and raises alerts when stock falls below configured thresholds. It is the authoritative owner of the `Ingredient`, `StockLevel`, `UsageRecord`, and `Alert` entities, and depends on the Menu Module and the Order Module through clearly defined peer service boundaries.

API Layer.

- **InventoryAPI** (*controller*) – receives HTTP requests, validates payloads, and delegates business logic to **InventoryService**. Exposes the REST endpoints: `listIngredients`, `createIngredient`, `updateStock`, `recordUsage`, `listLowStock`, `resolveAlert`.

Business Layer.

- **InventoryService** (*service*) – orchestrates ingredient lifecycle, stock adjustments, usage recording, and alert resolution, and delegates persistence to **InventoryRepository**. It invokes the peer service **MenuService** for menu-item resolution when usage is reconciled against served items, and **OrderService** to obtain the order events that drive automatic usage deduction. Exposes `updateStock`, `recordUsage`, `listLowStock`, and `resolveAlert`.

Peer Services (External Contracts).

- **MenuService** (*peer service contract*) – owned by the Menu Module and consumed here for menu-item lookup; provides `get_menu_item` and `list_items`. Drawn with a dashed border to indicate that it is referenced rather than implemented within this module.
- **OrderService** (*peer service contract*) – owned by the Order Module and consumed here for order-event listing; provides `list_events`. As with **MenuService**, it is a referenced contract, not an implementation belonging to this module.

Persistence Layer.

- **InventoryRepository** (*interface*) – defines the persistence contract: `listIngredients`, `getIngredient`, `saveIngredient`, `saveUsage`, `listAlerts`, `saveAlert`. Decouples the business layer from any specific storage technology, in line with the Dependency Inversion Principle.
- **SqlAlchemyInventoryRepository** (*adapter*) – implements **InventoryRepository**; backed by an asynchronous SQLAlchemy session (**AsyncSession**). Manages the lifecycle of **Ingredient**, **StockLevel**, **UsageRecord**, and **Alert** entities through a *composition* relationship (0..* for each), reflecting that the repository is the authoritative owner of persisted inventory records.

Domain Entities.

- **Ingredient** (*persisted*) – master record for a raw ingredient. Fields: **id**: UUID (primary key), **name**: String, **unit**: String (unit of measurement, e.g. kg, l, piece), **currentStock**: Decimal (canonical on-hand quantity), **lowThreshold**: Decimal (quantity below which a low-stock alert is raised).
- **StockLevel** (*persisted*) – timestamped snapshot of an ingredient’s stock level, used for trend analysis and reconciliation against **UsageRecord** entries. Fields: **id**: UUID (primary key), **ingredientId**: UUID (foreign key to **Ingredient**), **onHand**: Decimal, **updatedAt**: DateTime.
- **UsageRecord** (*persisted*) – log entry recording the consumption of an ingredient. Fields: **id**: UUID (primary key), **ingredientId**: UUID (foreign key to **Ingredient**), **qty**: Decimal, **usedAt**: DateTime, **usedByOrderId**: UUID? (nullable foreign key to an **Order**; populated when consumption is attributable to a specific order, and left null for manual adjustments such as wastage).
- **Alert** (*persisted*) – low-stock notification raised when an ingredient crosses its threshold. Fields: **id**: UUID (primary key), **ingredientId**: UUID (foreign key to **Ingredient**), **severity**: AlertSeverity, **createdAt**: DateTime, **resolvedAt**: DateTime? (nullable; populated when the alert is closed by a manager via the **resolveAlert** endpoint).

Enumerations.

- **AlertSeverity** (*enumeration*) – enumerates the severity levels assigned to low-stock alerts according to how far the on-hand quantity has fallen below the threshold: LOW, MEDIUM, HIGH, CRITICAL.

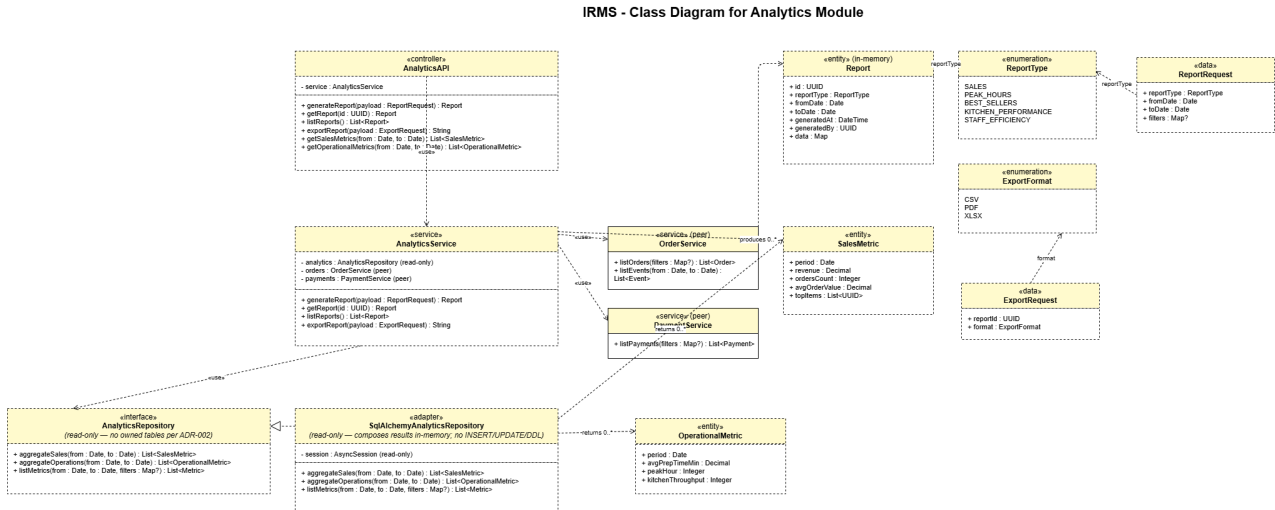


Figure 9: Class diagram for Analytics Module.

Analytics Module The Analytics Module produces sales and operational reports from data owned by other modules.

API Layer.

- **AnalyticsAPI** (*controller*) – receives HTTP requests, validates payloads, and delegates business logic to **AnalyticsService**. Exposes the REST endpoints: **generateReport**, **getReport**, **listReports**, **exportReport**, **getSalesMetrics**, **getOperationalMetrics**.

Business Layer.

- **AnalyticsService** (*service*) – orchestrates the report-generation pipeline. It queries the read-only **AnalyticsRepository** for pre-aggregated metrics, then invokes the peer services **OrderService** and **PaymentService** to obtain line-level transactional data, and finally composes the resulting **Report** value in memory. It exposes **generateReport**, **getReport**, **listReports**, and **exportReport**.

Peer Services (External Contracts).

- **OrderService** (*peer service contract*) – exposed by the Order Module and consumed here for order listing and event streaming, supplying the raw transactional data from which sales metrics are derived. Drawn with a dashed border to indicate that it is referenced rather than implemented within this module.

- **PaymentService** (*peer service contract*) – exposed by the Payment Module and consumed here for payment listing, supplying the revenue-side inputs to sales aggregations. As with **OrderService**, it is a referenced contract, not an implementation belonging to this module.

Persistence Layer (Read-Only).

- **AnalyticsRepository** (*interface, read-only*) – defines the read-only query contract: **aggregateSales**, **aggregateOperations**, **listMetrics**. This interface owns no tables; it issues only SELECT queries against tables owned by other modules.
- **SqlAlchemyAnalyticsRepository** (*adapter, read-only*) – implements **AnalyticsRepository**; backed by an asynchronous SQLAlchemy session restricted to read access. It executes only SELECT queries and composes the results in memory, returning **SalesMetric** (0..*) and **OperationalMetric** (0..*) as in-memory aggregates.

Domain Entities.

- **Report** (*in-memory*) – value object representing a generated report. Fields: **id**: UUID, **reportType**: ReportType, **fromDate**: Date, **toDate**: Date, **generatedAt**: DateTime, **generatedBy**: UUID (foreign key to the requesting **Staff** record in the Administration Module), **data**: Map (report-type-specific payload). Computed on demand and never written back to storage, consistent with the module’s read-only responsibility.
- **SalesMetric** (*in-memory aggregate*) – aggregated sales read-model produced by the repository. Fields: **period**: Date, **revenue**: Decimal, **ordersCount**: Integer, **avgOrderValue**: Decimal, **topItems**: List<UUID> (foreign keys to **MenuItem** records in the Menu Module).
- **OperationalMetric** (*in-memory aggregate*) – aggregated operational read-model. Fields: **period**: Date, **avgPrepTimeMin**: Decimal, **peakHour**: Integer, **kitchenThroughput**: Integer.

Enumerations.

- **ReportType** (*enumeration*) – enumerates the report categories supported by the module, each driving a distinct aggregation path: **SALES**, **PEAK_HOURS**, **BEST_SELLERS**, **KITCHEN_PERFORMANCE**, **STAFF_EFFICIENCY**.
- **ExportFormat** (*enumeration*) – enumerates the file formats supported by the export endpoint: **CSV**, **PDF**, **XLSX**.

Data Transfer Objects.

- **ReportRequest** – payload for report generation (**reportType**: ReportType, **fromDate**: Date, **toDate**: Date, **filters**: Map?). The optional **filters** map carries report-type-specific parameters such as station identifiers or staff identifiers.
- **ExportRequest** – payload for exporting an existing report (**reportId**: UUID, **format**: ExportFormat), used by the **exportReport** endpoint to render a previously generated report into the requested file format.

2.3.2 Component Diagram

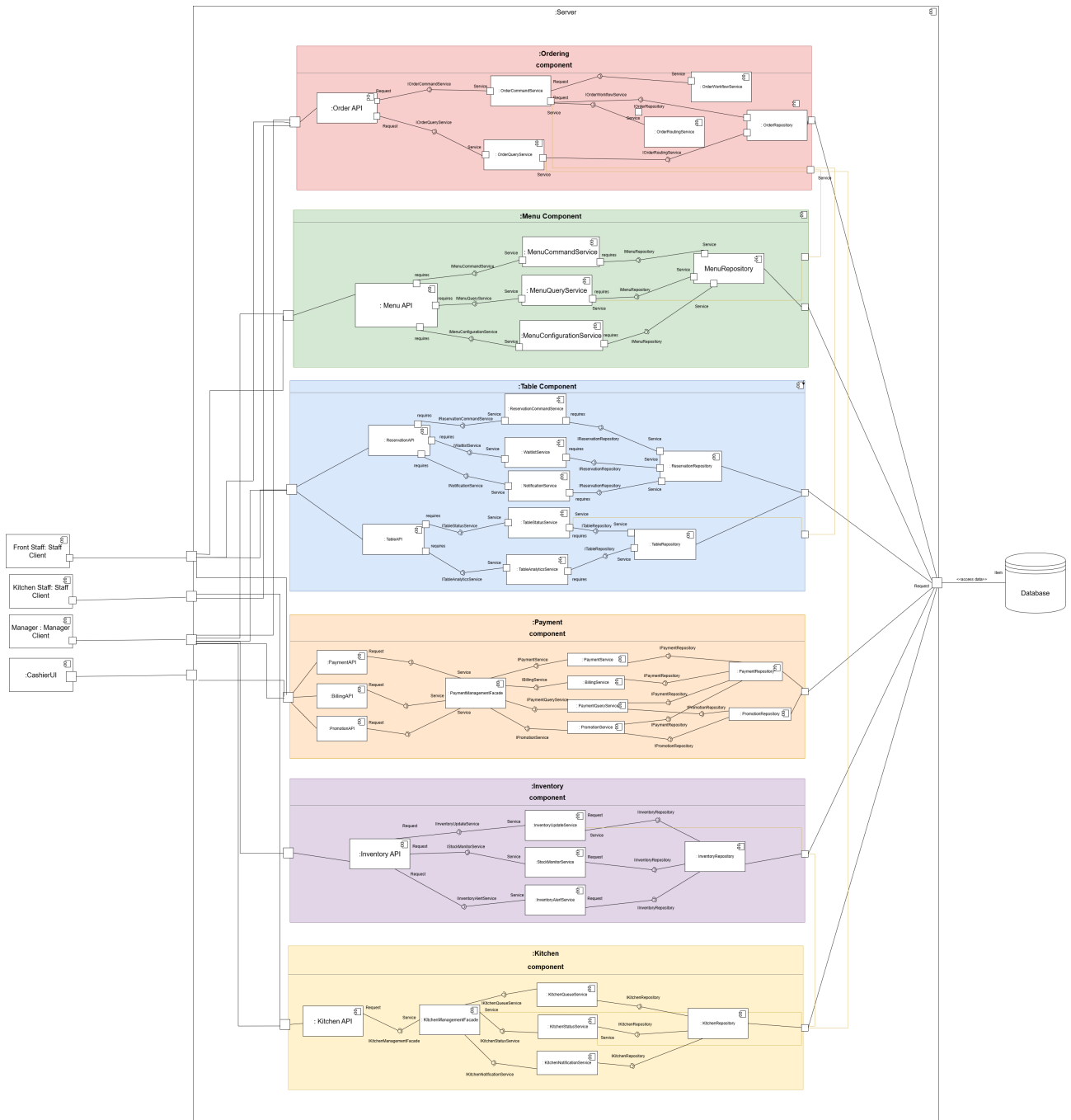


Figure 10: Component Diagram of the Restaurant Management System

1. Overview

The component diagram in Figure 10 describes the internal structure of the Intelligent Restaurant Management System (IRMS). The system is designed for internal restaurant operations and is accessed by staff-based user interfaces instead of direct customer users.

- **Front Staff UI:** handles ordering, table operations, and customer-facing tasks.
- **Kitchen Staff UI:** supports kitchen order preparation and progress updates.
- **Cashier UI:** handles billing, payment, refund, and split-bill operations.
- **Manager UI:** monitors menu, inventory, reports, users, and operational data.

2. No API Gateway

The system does not use a centralized API Gateway. Instead, each role-based UI communicates directly with the required domain controller. This keeps the architecture simpler and avoids placing all request routing logic into one central gateway.

- *OrderController*: handles order-related requests.
- *MenuController*: handles menu and dish-related requests.
- *PaymentController*: handles payment, billing, refund, tip, and split-bill requests.
- *InventoryController*: handles inventory and stock-related requests.
- *KitchenController*: handles kitchen queue and preparation progress requests.
- *TableController*: handles table, reservation, and waitlist requests.
- *ReportController*: handles analytics and report requests.
- *AdminController*: handles administrative and user-role requests.

3. Common Component Structure

Each domain component follows a layered structure. This structure separates request handling, business processing, and data access responsibilities.

- **Controller Layer**: receives requests from staff UIs and maps them to business services.
- **Business Layer**: contains services that implement business workflows and validation rules.
- **Persistence Layer**: contains repositories or DAOs for database access.
- **Shared DTOs**: define data contracts used between components and layers.

The general communication flow is:

- UI sends a request to a domain controller.
- Controller delegates the request to one or more services.
- Services apply business rules and coordinate workflow logic.
- Repositories read or write data from the shared database.
- DTOs are used to transfer structured data between layers.

4. Domain Components

The system is divided into domain-based components. Each component represents one major business capability of the restaurant system.

(a) *Order Management*

The Order Management Component handles the full order lifecycle from creation to completion or cancellation.

- **Controller**: *OrderController*
- **Main services**: Order Creation, Order Modification, Order Routing, Order Completion, Order Cancellation, Order Query, and Order Audit services.
- **Repositories**: Order Repository, Menu Repository, and Audit Repository.
- **Main responsibilities**: create orders, update orders, route orders, complete orders, cancel orders, and record order logs.

(b) *Menu Management*

The Menu Management Component manages menu items, dish categories, menu configuration, and promotion information.

- **Controller**: *MenuController*
- **Main services**: Menu Update, Menu Configuration, Menu Query, and Promotion Query services.
- **Repositories**: Menu Repository and Promotion Repository.
- **Main responsibilities**: update menu items, configure menu display, retrieve menu catalog, and query active promotions.

(c) *Payment Management*

The Payment Management Component handles payment workflows, including billing, payment completion, refund processing, tip calculation, and split-bill operations.

- **Controller:** *PaymentController*
- **Main services:** Payment Creation, Refund, Split Bill, Payment Completion, Bill Calculation, Tip, Payment Query, and Payment Audit services.
- **Repositories:** Payment Repository, Order Repository, Menu Repository, and Audit Repository.
- **Main responsibilities:** create payments, calculate bills, process refunds, split bills, add tips, complete payments, and store payment logs.

(d) ***Inventory Management***

The Inventory Management Component manages stock information and supports inventory monitoring.

- **Controller:** *InventoryController*
- **Main services:** Inventory Update, Historical Usage, Notification Alert, and Inventory Query services.
- **Repository:** Inventory Repository.
- **Main responsibilities:** update stock, record stock movements, query inventory, track historical usage, and send low-stock alerts.

(e) ***Kitchen Management***

The Kitchen Management Component supports kitchen preparation workflows and order status updates.

- **Controller:** *KitchenController*
- **Main services:** Order Retrieval, Order Organization, Progress Update, and Notification Alert services.
- **Repositories:** Order Repository and Dishes Repository.
- **Main responsibilities:** retrieve pending orders, organize kitchen queues, update preparation progress, and notify staff when dishes are ready.

(f) ***Table Management***

The Table Management Component handles reservations, table availability, waitlists, and customer notifications.

- **Controller:** *TableController*
- **Main services:** Reservation Creation, Schedule Booking, Waitlist Request, Reservation Completion, Wait Time Estimation, Customer Notification, and Table Service Time services.
- **Repositories:** Table Repository and Customer Repository.
- **Main responsibilities:** create reservations, manage table status, handle waitlists, estimate waiting time, and notify customers.

(g) ***Analytics & Reports Management***

The Analytics and Reports Management Component provides reporting and operational analysis for managers.

- **Controller:** *ReportController*
- **Main services:** Sales Report, Operational Analytics, Export Report, and Audit Analysis services.
- **Repository:** Audit Repository.
- **Main responsibilities:** generate sales reports, analyze operations, export reports, and summarize audit data.

(h) ***Administrative Tools***

The Administrative Tools Component supports system administration, user-role management, and promotion control.

- **Controller:** *AdminController*
- **Main services:** Promotion Creation, User Role Management, Audit Log Management, and Account Query services.
- **Repositories:** Promotion Repository, Account Repository, and Audit Repository.
- **Main responsibilities:** manage promotions, assign roles, revoke roles, query accounts, and review audit logs.

5. Shared Libraries

Shared Libraries provide common DTOs and data contracts used by multiple components. This helps the system avoid duplicated model definitions and keeps communication consistent.

- **Common DTOs:** Audit DTO and Account DTO.
- **Order DTO:** used for order-related communication.
- **Menu DTOs:** Menu Catalog DTO, Dishes DTO, and Promotion DTO.
- **Table DTOs:** Table DTO and table status-related data.
- **Payment DTOs:** Payment Catalog DTO and billing-related data.
- **Inventory DTOs:** Inventory Catalog DTO and stock-related data.

6. Shared Database

The system uses a shared database approach. All domain components access the same logical database through their own repositories, while controllers and services do not access the database directly.

- Repositories are responsible for all data access operations.
- The shared database stores orders, menu items, tables, reservations, payments, inventory, accounts, reports, and audit logs.
- This approach keeps operational data centralized and consistent.
- It is suitable for the current system scale because it avoids distributed database complexity.

7. Design Rationale

Overall, the component diagram follows a domain-based modular design. Each major business function has a clear controller, business services, repositories, DTOs, and database access responsibility.

- Domain components improve separation of concerns.
- Controllers provide clear entry points for each domain.
- Services keep business logic independent from request handling.
- Repositories isolate database access from business logic.
- Shared DTOs keep communication consistent.
- The shared database simplifies data consistency and transaction handling.
- Removing the API Gateway reduces complexity for a medium-sized internal system.

2.3.3 Allocation View

The Allocation View describes how the IRMS software components are mapped onto the physical infrastructure. It illustrates the runtime environment, including devices, server clusters, and communication protocols. This view is presented through the Deployment Diagram, which shows how UI clients interact with the Application Server Cluster and how services are deployed as execution units within the system.

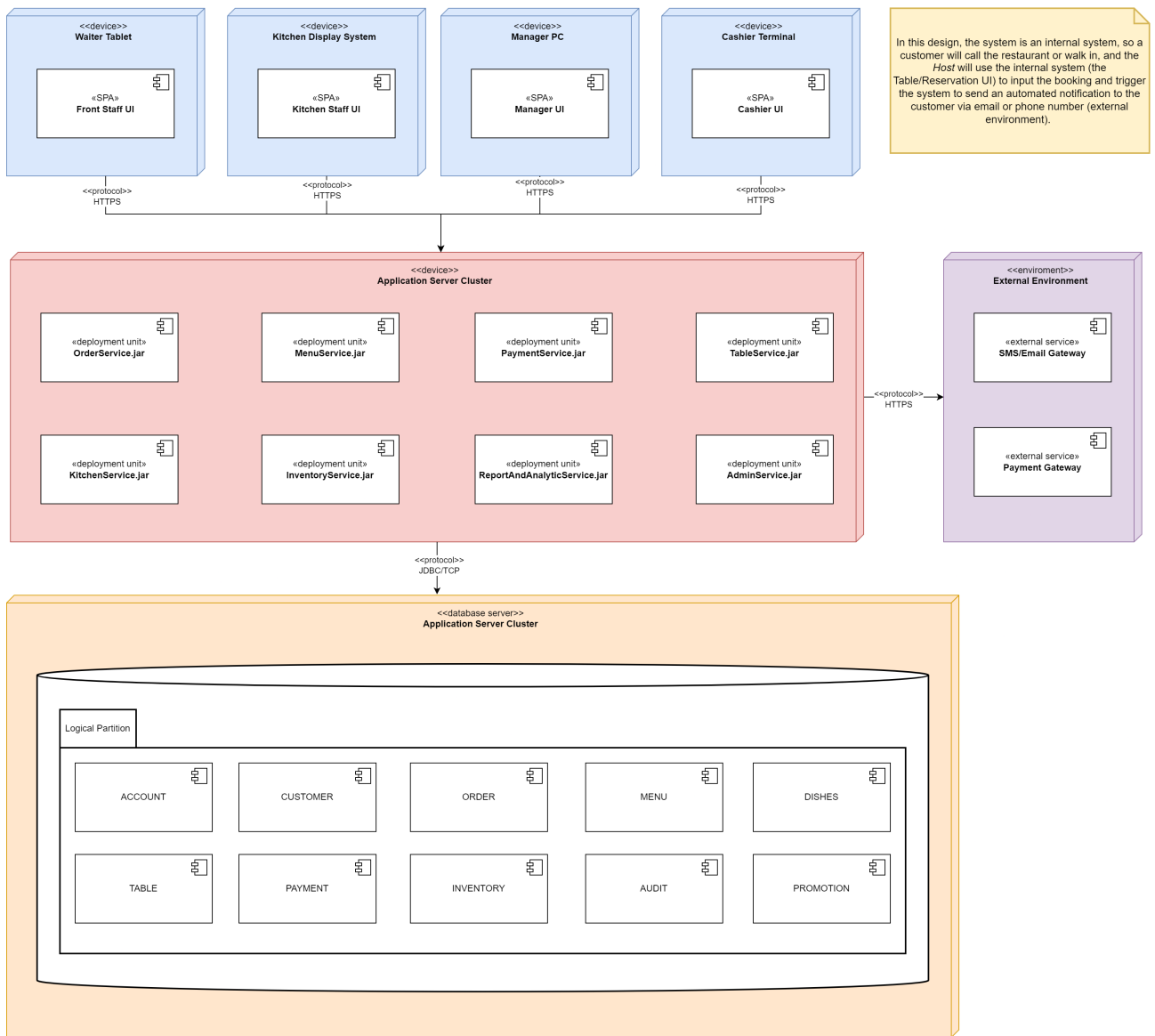


Figure 11: **Deploy Diagram** for IRMS system

The Deployment Diagram includes three main elements:

- **Client Devices and User Interfaces**

The system is accessed through multiple role-based client devices, each running a Single Page Application (SPA):

- Waiter Tablet – Front Staff UI
- Kitchen Display System – Kitchen Staff UI
- Manager PC – Manager UI
- Cashier Terminal – Cashier UI

All client devices communicate with the backend using the HTTPS protocol, ensuring secure and standardized interaction.

- **Application Server Cluster**

All backend services are deployed within a centralized *Application Server Cluster*. Each service is packaged and deployed as an independent deployment unit (e.g., *OrderService.jar*, *KitchenService.jar*, *AdminService.jar*).

The cluster is responsible for processing requests from client applications and routing them to the appropriate service internally. This design provides scalability, simplifies deployment, and centralizes system management.

• External Systems and Database Server

The Application Server Cluster interacts with external systems and the database layer:

– External Environment (HTTPS):

- * SMS/Email Gateway – used for sending notifications to customers
- * Payment Gateway – used for processing digital payments

– Database Server (JDBC/TCP):

The system uses a centralized database server with logically partitioned data domains, including: *Account, Customer, Order, Menu, Dishes, Table, Payment, Inventory, Audit, and Promotion.*

All services access the database through standardized database protocols, ensuring consistency and integrity of data across the system.

Overall, the Allocation View demonstrates a centralized deployment model where multiple client interfaces interact with a clustered backend and shared database, while also integrating with external services for communication and payment processing.

2.3.4 Entity-Relationship Diagram

The ERD in Figure 12 captures the data structures of the IRMS, with focus on the Ordering domain and its surrounding context. The diagram exposes nine entities partitioned across the service ownership boundaries used in the system design.

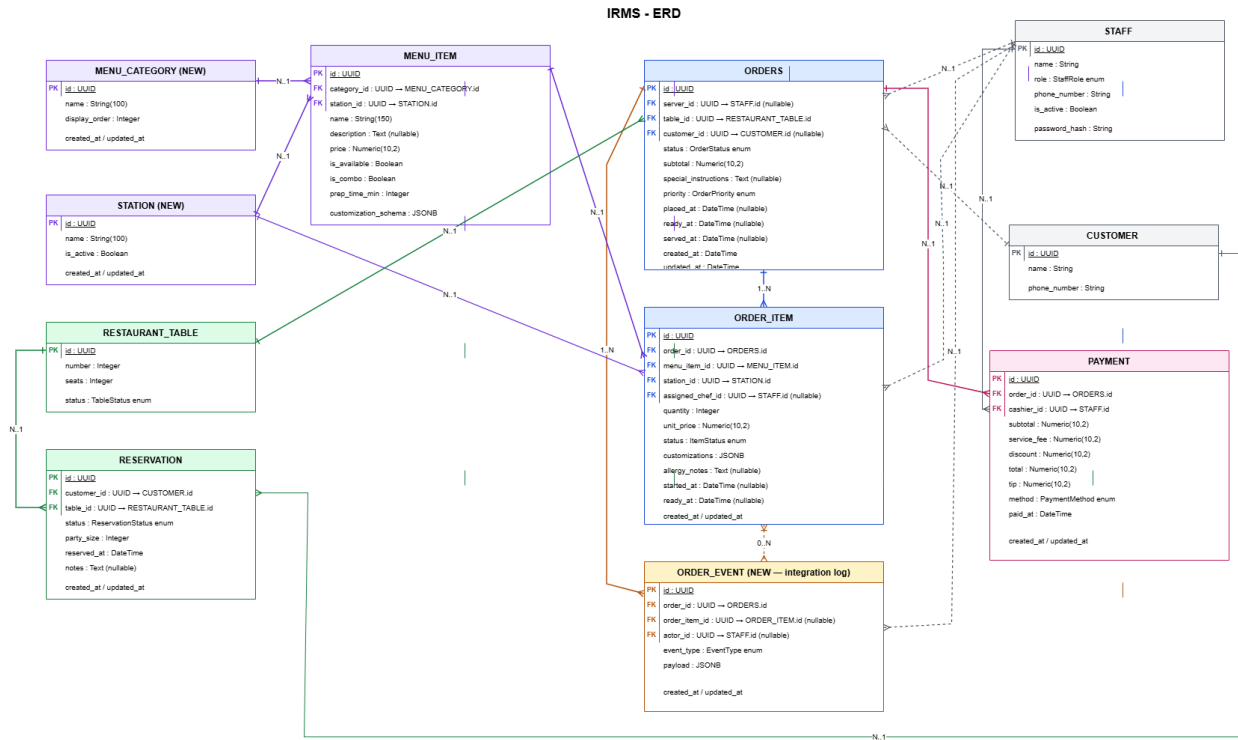


Figure 12: IRMS Entity-Relationship Diagram

Ordering service owns three entities:

- **ORDERS** – captures the order lifecycle from placement through preparation and service, with timestamps for each transition (`placed_at`, `ready_at`, `served_at`).
- **ORDER_ITEM** – carries its own pricing snapshot (`unit_price`), workflow timestamps (`started_at`, `ready_at`), and customisation payload (`customizations`, `allergy_notes`), allowing orders to remain consistent and billable independently of menu changes after submission.
- **ORDER_EVENT** – an append-only audit log that records lifecycle transitions for traceability, debugging, and compliance; written exclusively by the Ordering service and read only by Analytics and Administration.

Menu service owns three entities:

- **MENU_ITEM** – the menu catalogue including price, availability, preparation time, and customisation schema.
- **MENU_CATEGORY** – groups menu items into displayable categories with ordering metadata.
- **STATION** – kitchen station for routing items to the correct preparation area.

Categories and stations are modelled as first-class entities rather than free-text fields, which keeps menu evolution decoupled from item records and supports referential integrity.

Table service owns two entities:

- **RESTAURANT_TABLE** – the physical table inventory with seating capacity and current status.
- **RESERVATION** – links customers to tables with timing and party-size metadata.

Payment service owns one entity:

- **PAYMENT** – bill calculation, payment processing, and cashier attribution; references **ORDERS** and **STAFF** for traceability.

Administration service owns two entities:

- **STAFF** – staff records including role, contact details, and authentication credentials.
- **CUSTOMER** – customer reference data with contact information.

Both entities are consumed read-only by peer services as shared reference data.

Cross-service foreign keys (`server_id`, `cashier_id`, `customer_id`, `assigned_chef_id`) preserve referential integrity at the database level. Write authority remains with the owning service: peer services may follow these references for reads but never mutate another service’s tables directly. This boundary is enforced at the application layer rather than the schema, allowing ACID transactions to remain available within each service while keeping ownership lines clean for any future migration to per-service databases.

2.4 Architecture Decisions

The IRMS adopts a **Service-Based Architecture** as its overall structural style. The system is decomposed into coarse-grained domain services—Ordering, Kitchen Display (KDS), Menu, Table, Payment, Inventory, Analytics, and Administration—each owning a complete vertical slice of its responsibility (API surface, business logic, persistence boundary). Services communicate through stable API contracts; one service may not reach directly into another service’s internal logic or data.

Three decisions follow from that style choice and govern how IRMS must be constructed: how the system is partitioned, how data ownership is enforced, and how clients interact with the system.

Service-Based Architecture with Domain Partitioning

Context. The IRMS supports several operational areas with different actors and rates of change: order taking, kitchen workflow, table management, billing, and others. A monolithic codebase couples all domains, so every change risks the whole system. A full microservices deployment introduces orchestration, distributed tracing, and per-service infrastructure that the team cannot staff at this scope.

Alternatives considered.

- **Monolithic architecture** – simple to build but couples every domain together; one change risks the whole system.
- **Microservices** – maximum independence, but disproportionate operational overhead for a single-restaurant system with five user roles.
- **Service-Based Architecture (chosen)** – a pragmatic middle ground: coarse-grained domain services with clear contracts, far more modular than a monolith and far simpler to operate than microservices.

Decision. Partition the system by *business capability*. Each domain becomes a service that owns its API surface, its workflows, and its data. Peers access a service only through its published contract; internal logic stays hidden. Cross-domain workflows are coordinated through each service’s published API; one service never reaches into another’s internal classes, data, or storage.

Consequences.

- **Positive.** Clear ownership; each domain’s business logic evolves at its own pace; targeted testing; a clean migration path to fully distributed deployment if scale ever demands it.
- **Negative.** Extra structural discipline up front; cross-domain workflows must go through explicit integration mechanisms.
- **Trade-off.** Faster per-domain delivery, paid for with the cost of enforcing service boundaries in code review until those boundaries can be enforced automatically.

Shared Database with Logical Service Ownership

Context. Each service still needs persistence. Two extremes exist: a fully shared schema where any service may read or write any table, or a strict database-per-service split where each service has its own physical database.

Alternatives considered.

- **Shared schema, free-for-all access** – easy to query across domains but couples services through the database; any schema change becomes a coordination event for the whole system.
- **Strict database-per-service** – strongest isolation, but multiplies infrastructure and breaks ACID transactions across related entities.
- **Single shared database with logical ownership (chosen)** – the default topology described by Richards & Ford for service-based architecture: one physical database, but each service is the *sole writer* of a defined table set.

Decision. All services persist to one shared database. Within it, table ownership is partitioned by service:

- **Ordering** – orders, order items.
- **Menu** – categories, menu items.
- **Table** – tables, reservations.
- **Payment** – payments, receipts.
- **Administration** – staff records.
- **Inventory** – ingredients, stock movements (planned).
- **Kitchen Display** – consumer of Order data; no owned tables in current iteration.
- **Analytics & Reports** – read-only consumer; no owned tables.

A service may *read* data outside its tables only by calling the owning service's API. A service may never write to, or run schema-modifying operations against, another service's tables. Cross-service foreign keys may exist for referential integrity at the database level, but write authority remains with the owning service at the application layer.

Consequences.

- **Positive.** ACID transactions remain available within a service; cross-domain coupling becomes explicit and reviewable; the table-ownership lines already match the service boundaries, so a future migration to per-service databases requires no schema rework.
- **Negative.** Cross-service queries cannot use a single SQL join; they must be composed at the application layer.
- **Trade-off.** The query convenience of a shared schema is exchanged for clean ownership boundaries that survive any future scale-out.

Centralised API Layer as the Single Entry Point

Context. Different user roles—server, host, kitchen staff, cashier, manager—interact with different combinations of services. Exposing each service directly to clients leaks the service topology and embeds routing logic in every UI. Cross-cutting concerns such as authentication, logging, and rate limiting also need a stable home.

Alternatives considered.

- **Per-service exposure** – each UI knows every service's endpoint; a leaky abstraction that hard-codes the service map into clients.
- **Centralised API layer (chosen)** – a single addressable surface, recommended by Richards & Ford as the natural front-end for a service-based architecture, that routes requests to the correct domain service.

Decision. All UI clients (Front Staff, Kitchen, Cashier, Manager) talk to one API surface. That surface routes each request to the owning domain service and is the single home for cross-cutting concerns: authentication, role-based access control, rate limiting, request logging, and audit. No service is reachable from clients except through this layer. Services expose only Data Transfer Objects (DTOs)—neutral data structures that carry information across the boundary without exposing internal entities or storage representations.

Authentication and role-based access control are scoped as middleware concerns at this layer. Their full implementation is deferred for this iteration; the architecture is designed to accept them as additions to the API layer without changes to the domain services.

Consequences.

- **Positive.** Clients stay simple; cross-cutting policy lives in one place and changes deploy once; service implementations remain interchangeable behind a stable DTO contract.
- **Negative.** The API layer is a single point on the request path; if it fails, the system is unreachable.
- **Trade-off.** The simplicity of a single entry point and a single home for cross-cutting policy is worth the operational risk, which is mitigated by keeping the layer thin and stateless.

2.5 Design Principles

The following principles guide how the IRMS is designed and implemented. They are based on the system architecture and the main goals such as performance, usability, and maintainability.

1. Keep business logic inside the Service layer

Each module (e.g., Order, Kitchen, Table) follows the same structure: Router → Service → Repository. The Router only handles HTTP requests, and the Repository only accesses the database.

e.g. order validation and status transitions (e.g., `QUEUED` → `PREPARING` → `READY`) are implemented in *KitchenService*, not in the API layer. This makes the system easier to test and debug.

2. Organize the system by business domain

The system is divided into services such as Ordering, Kitchen, Payment, and Table Management. Each service handles one main responsibility.

e.g. the Kitchen service only deals with order display and progress updates, while Payment service handles billing and transactions. This avoids mixing unrelated logic in the same module.

3. Respect data ownership even with a shared database

Although all services use the same database, each service owns its own tables.

e.g. only the Order service can update the *ORDER* table. If another service (e.g., Kitchen) want to update order data, it must call the Order service instead of querying the table directly.

This prevents hidden dependencies and keeps business rules consistent.

4. Use a single API entry point for all clients

All UIs (Front Staff, Kitchen, Cashier, Manager) send requests to one API layer.

This avoids exposing internal service structure to the frontend. It also allows features like logging, authentication, and request validation to be handled in one place instead of repeating in every service.

5. Use DTOs for all data exchange

Services and APIs communicate using DTOs from shared libraries.

e.g. *OrderDTO* is used when sending order data to the Kitchen UI or Payment service. The database schema is never exposed directly.

This allows internal changes (e.g., database refactoring) without affecting other modules.

6. Optimize for fast response in critical workflows

Operations like placing an order or updating kitchen status must be fast.

e.g. when a waiter submits an order, it should appear immediately on the Kitchen Display System. This is handled by keeping service logic simple and avoiding unnecessary processing in the request path.

7. Ensure data is saved immediately for critical actions

Important actions such as order creation and payment must be stored in the database right away.

Each request uses a single transaction, so if something fails, the whole operation is rolled back. This avoids partial data (e.g., order created but payment missing).

8. Design services to scale during peak hours

The system is deployed in an application server cluster, where multiple instances of services can run.

Since services do not store session state, new instances can be added during busy periods (e.g., lunch rush) without changing the code.

9. Design UI based on staff roles and tasks

Each UI is built for a specific role:

- Front Staff UI → order taking
- Kitchen UI → order processing
- Cashier UI → payment
- Manager UI → management

e.g. the Kitchen UI only shows dishes and status updates, not billing information. This reduces complexity and helps staff work faster with fewer mistakes.

10. Keep the codebase consistent and easy to change

All modules follow the same structure and coding style. New features (e.g., promotions or delivery) can be added as new services or modules without affecting existing ones too much.

e.g. adding a Loyalty module would follow the same Router–Service–Repository pattern, making it easier for developers to understand and implement.

3 Core Implementation

The complete source code is available at [GitHub Repo](#).

3.1 Architectural Approach

The backend is implemented as a service-based monolith using **FastAPI** over an asynchronous **SQLAlchemy** engine. Each business capability lives in its own module under `backend/app/modules/` and follows a uniform three-layer structure.

- *Router* – a thin HTTP layer that deserialises requests, calls a service via dependency injection, and returns DTOs. The router contains no business logic of its own.
- *Service* – one concrete class per module that holds the business rules: state-machine guards, validation, cross-module coordination, and event emission.
- *Repository* – a Protocol declaring the persistence contract that the service depends on, alongside a SQLAlchemy adapter that implements it. The repository is the only layer that touches the ORM.

Three design rules govern how the modules fit together. First, each module logically owns its tables: another module may read cross-module data only by calling the owner’s public service methods, and may never write to the owner’s tables directly. This keeps data ownership unambiguous and survives any future migration to per-module databases. Second, services depend on each other through their published method surface rather than through extracted interfaces, because the peers live in the same deployable and no second implementation is foreseen. Third, all modules are mounted behind a single API surface at `/api/v1/*`, where cross-cutting concerns – CORS, request logging, future authentication and access control – live in one place rather than being duplicated across module routers.

Dependency injection is wired in a single composition root. Every request opens a fresh asynchronous session and binds repositories and services to it, so all writes within one HTTP request share a single transaction.

3.2 Backend Module Structure

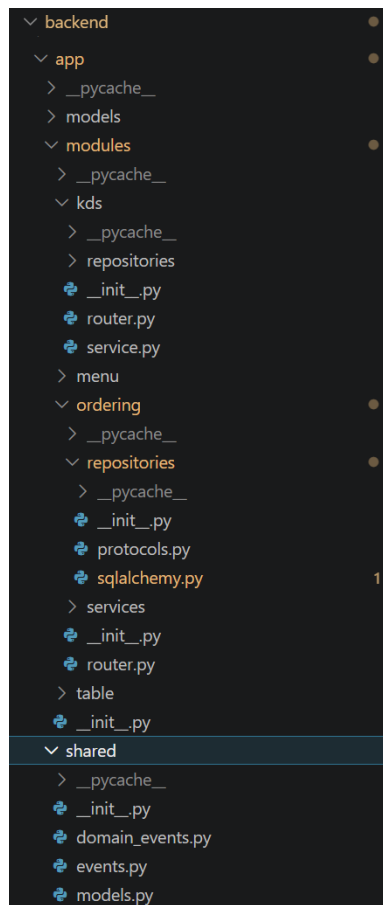


Figure 13: Backend Structure

3.2.1 Order Module

The Order module is the central backend module of the system. It owns the entire order lifecycle from initial draft through closure, and is the sole writer to the `order`, `order_item`, and `order_event` tables. Every other module that needs order data – KDS, Inventory, Analytics, Payment – reads it through this module’s service rather than touching the tables directly.

Responsibility.

- Records and persists every order placed in the restaurant, with its constituent items and any customisations or allergy notes.
- Enforces the legal sequence of order states, rejecting any transition that the current state does not permit.
- Produces an immutable audit trail of every meaningful state change, queryable by other modules without coupling them to internal order tables.
- Coordinates with the Menu and Table modules at order-creation time to validate menu-item identifiers and table assignments.

Service contract. The service exposes four groups of operations.

- *Read operations* return orders, individual orders, and the event log.
- *Item-management operations* add, update, or remove items on an order, but only while the order is still in DRAFT state.
- *Lifecycle transitions* – submit, serve, close, and cancel – move an order through its state machine.
- *Kitchen coordination methods* are called by the KDS module to update item status, promote an order to ready, or recall a completed order back to the kitchen queue.

State machine. The order moves through six lifecycle stages: **DRAFT**, **PLACED**, **IN_KITCHEN**, **READY**, **SERVED**, and **CLOSED**, with **CANCELLED** reachable from any non-terminal state. Individual items move through their own shorter sequence: **QUEUED**, **PREPARING**, **READY**, **SERVED**, with **VOIDED** as an alternate exit. Each transition method begins with an explicit state guard, so an attempt to serve an order that has not yet been marked ready is rejected before any state is mutated. Promotion to **READY** is idempotent, which makes the corresponding endpoint safe to retry under network failure.

Audit-trail event log. Every meaningful state change is appended to the event log as an **OrderEvent** with one of five types: **PLACED**, **ROUTED**, **STATUS_CHANGED**, **RECALLED**, or **CANCELLED**. The log is exposed through a public service method and gives downstream modules – Analytics for sales reporting, Inventory for usage reconciliation – a stable read seam that does not depend on internal table structure.

REST endpoints. All routes are mounted under `/api/v1/orders` and fall into four groups.

- *List and fetch* – retrieve all orders or a single order by identifier.
- *Order creation* – create a new draft order against a table.
- *Item management* – add, update, or remove items on a draft order.
- *Lifecycle transitions* – submit, cancel, serve, or close an order.

3.2.2 KDS Module

The Kitchen Display System (KDS) module presents the kitchen-facing view of in-progress orders and gives the kitchen staff the controls needed to drive items through preparation. It owns only the **station** table, which records the kitchen stations to which items are routed; it owns no order data of its own. Every write that affects an order or its items is delegated to the Order module’s published service rather than performed directly against order tables.

Responsibility.

- Manages the catalogue of kitchen preparation stations – grill, salad, bar, and so on.
- Composes a per-station ticket queue showing items currently awaiting preparation at that station.
- Allows kitchen staff to advance individual item status – queued, preparing, ready – and to bump finished items off the screen.
- Promotes an order to **READY** automatically once every one of its items has been marked ready by the kitchen.
- Allows recall of a completed order back to the kitchen queue when a correction is needed.

Boundary with the Order module. The module’s most important architectural property is that it never touches order tables directly. Every item-status write, every order-level promotion, and every recall flows through a public method on the Order service. The kitchen never opens a database session against order data of its own. This is what keeps the data-ownership rule – one module, one table set, one writer – enforced in code rather than only in convention. A code reviewer can verify the property mechanically by searching the KDS module for any reference to the order tables; the search returns nothing.

KitchenTicket as a composed view. A kitchen ticket is not a persisted entity. It is a transient DTO assembled at query time from three sources: the order itself (read through the Order service), the items belonging to the requested station, and the table number (read through the Table service). Because the ticket is recomputed on every request, the KDS module remains stateless with respect to order data, and there is no cache to invalidate when the underlying order changes.

Auto-promote pattern. When kitchen staff mark the last item of an order as ready, the KDS service does not require them to also promote the order itself. The item-status method checks whether every item on the affected order has reached **READY**; if so, it calls the Order service’s **transition_order_to_ready** method, which atomically transitions the order and appends a **STATUS_CHANGED** event. The kitchen UI stays simple – staff only ever bump individual items – while the order-level state machine remains owned by the Order module, where the order invariants live.

REST endpoints. All routes are mounted under `/api/v1/kds` and fall into five groups.

- Station CRUD – list and create kitchen stations.
- Ticket retrieval – fetch the ticket queue for a given station, optionally filtered by item status.
- Item-status writes – change an item’s status or bump a finished item off the screen.
- Order recall – send a completed order back to the kitchen queue.
- Event log – query the order event log for kitchen audit purposes.

3.3 Frontend Structure

3.3.1 Front Staff UI - Order Feature

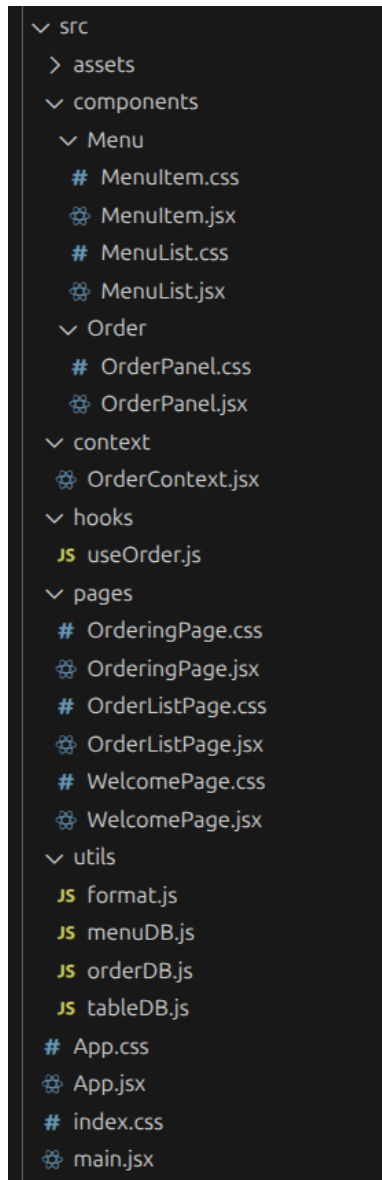


Figure 14: File structure of Front Staff UI - Order Feature

The frontend is implemented as a multi-page application using React and Vite. The source code is organized into multiple directories, each serving a specific purpose:

- **Component**

The `components/` directory contains reusable UI elements. For example, the `Menu/` module includes components such as `MenuItem` and `MenuList`, which render menu data and handle user interactions like selecting item, increase item, decrease item.

The `Order/` directory encapsulates components related to order functionality, such as `OrderPanel`, which displays the current order and allows users to add allergy notes or some note with specific dish.

- **Page**

The `pages/` directory defines the main application views, including `OrderingPage.jsx`, `OrderListPage.jsx`, and `WelcomePage.jsx`.

Each page composes multiple components to deliver complete user workflows.

For example, the Ordering Page integrates menu components and the order panel to support the order creation process.

- **State Management (Context + Hook)**

The application uses a unified state management approach combining React Context and custom hooks. The `OrderContext` defines a global state container that stores the current order (`orderItems`) and provides operations such as adding items, decreasing quantities, loading an existing order, and clearing the order.

The `OrderProvider` manages this state internally using `useState`, ensuring that all business logic related to order manipulation is centralized.

To simplify access, a custom hook (`useOrder`) is used as an abstraction layer. Components call this hook to retrieve both state and update functions instead of directly interacting with the context. This improves readability, reduces boilerplate, and ensures consistent usage across the application.

- **Utility**

The `utils/` directory provides helper functions and lightweight data access modules that encapsulate communication with backend APIs and local caching logic. It abstracts data retrieval and formatting from the UI, allowing components to remain focused on presentation.

Specifically, `menuDB.js` and `tableDB.js` implement asynchronous initialization functions (e.g., `initMenu()`, `initTables()`) that fetch data from REST endpoints and store it in module-level caches. These caches are then accessed through synchronous getter functions such as `getMenu()`, `getItemById()`, and `getListTables()`, reducing repeated API calls and improving performance.

Additionally, the utility includes data transformation logic. For example, menu items are enriched with visual metadata (e.g., emoji mapping based on item names) to enhance UI presentation. Formatting helpers such as `formatUSD` standardize the display of values across the application.

By isolating API interaction, caching, and data formatting within this layer, the frontend achieves better separation of concerns and remains decoupled from backend implementation details.

3.3.2 Kitchen Staff UI - KDS Feature

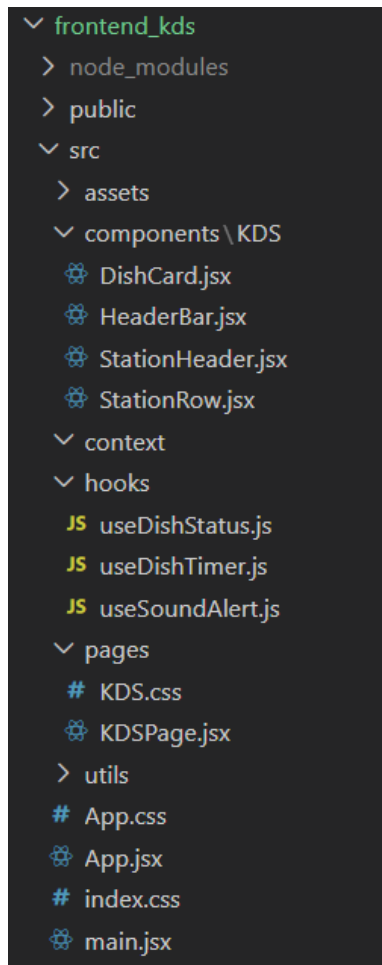


Figure 15: File structure of Kitchen Display System (KDS) UI

The Kitchen Display System (KDS) frontend is also implemented using React and Vite framework. It is designed to support real-time kitchen operations by visualizing active orders per station and providing interactive controls for kitchen staff.

The source code is organized into multiple directories, each responsible for a specific concern:

- **Component**

The `components/KDS/` directory contains UI components specifically designed for kitchen workflows.

- **StationRow**: Represents a single kitchen station (e.g., Grill, Fryer) and displays all dishes assigned to that station.
- **StationHeader**: Displays the station name and acts as a visual separator between stations.
- **DishCard**: The core unit of the KDS interface, representing an individual dish. It displays dish name, quantity, table number, order ID, allergy notes, preparation timer, and action buttons (e.g., Start, Ready).
- **HeaderBar**: Provides a global header for the KDS page, which may include system title or status indicators.

Unlike traditional order-based views, the KDS UI is designed around **dish-level granularity**, ensuring that kitchen staff can focus on preparing individual items rather than entire orders.

- **Page**

The `pages/` directory defines the main KDS interface through `KDSPage.jsx`.

This page is responsible for:

- Fetching station and ticket data from backend APIs.
- Organizing data into station-based groups.
- Rendering a list of `StationRow` components, each containing multiple dish cards.

The page also includes `KDS.css`, which defines styling for layout, including station columns, dish card appearance, and visual alerts (e.g., warning and expired states).

- **State Management (Hooks)**

The KDS feature adopts a lightweight state management approach using custom React hooks instead of a global context.

- `useDishTimer`: Manages countdown logic for each dish based on its `placed_at` timestamp and preparation time. It updates remaining time every second and determines whether a dish is expired.
- `useDishStatus`: Handles state transitions of a dish (e.g., `QUEUED` → `PREPARING` → `READY`) and manages UI states such as loading indicators during updates.
- `useSoundAlert`: Provides an audio feedback mechanism when a dish exceeds its preparation time. It generates a short alert sound using the Web Audio API.

Each hook encapsulates a specific piece of business logic, improving modularity and reusability across components.

- **Data Transformation**

Although the backend API returns data in a ticket-based structure, the frontend transforms this data into a dish-centric format.

Specifically, within `StationRow`, ticket data is flattened into a list of dishes:

- Each ticket contains multiple items.
- Each item is mapped into a dish object.
- Additional metadata (e.g., `table_number`, `order_id`, `placed_at`) is attached to each dish.

This transformation aligns the UI with real kitchen workflows, where tasks are performed per dish rather than per order.

- **Utility**

The `utils/` directory (if utilized) is intended to encapsulate helper functions such as API calls, data formatting, or shared logic. While minimal in the KDS module, this layer supports separation of concerns and future extensibility.

Overall, the KDS frontend follows a modular and component-driven architecture, emphasizing real-time responsiveness, clear separation of concerns, and alignment with practical kitchen operations.

3.4 Implement Features

3.4.1 Order Feature

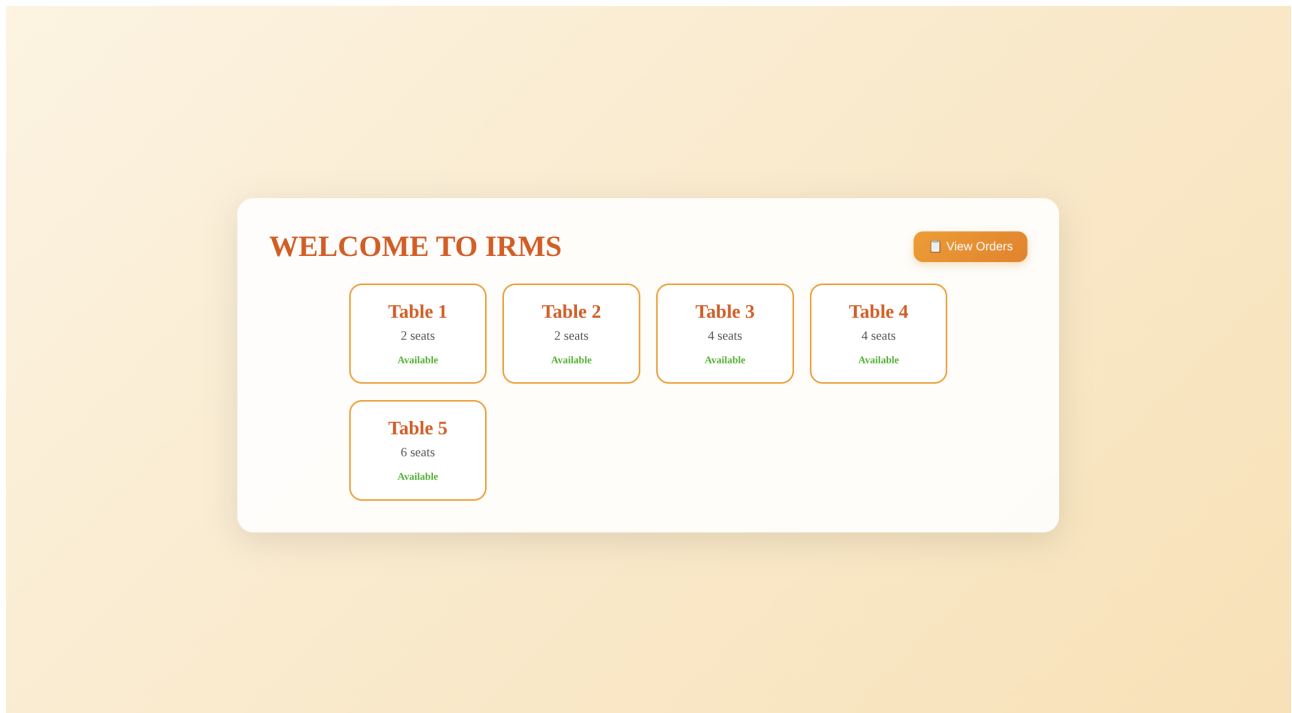


Figure 16: Welcome Page for Ordering

First, staff are directed to the welcome page to select a table for placing an order. This page serves as the entry point of the ordering workflow <http://localhost:5173/>.

- The system retrieves table data from the backend using the utility function `initTables()`, which asynchronously calls the API endpoint (`/api/v1/tables`).
- The fetched data is stored in a module-level cache, allowing reuse without repeated API calls.
- After initialization, the function `getListTables()` is used to synchronously access the cached table data.
- The UI renders a list of tables along with their current status (e.g., occupied or available), helping staff quickly identify and select a table.

When a table is selected, the system proceeds to the ordering page `/create-order?tableId=${tableId}` to begin the order creation process.

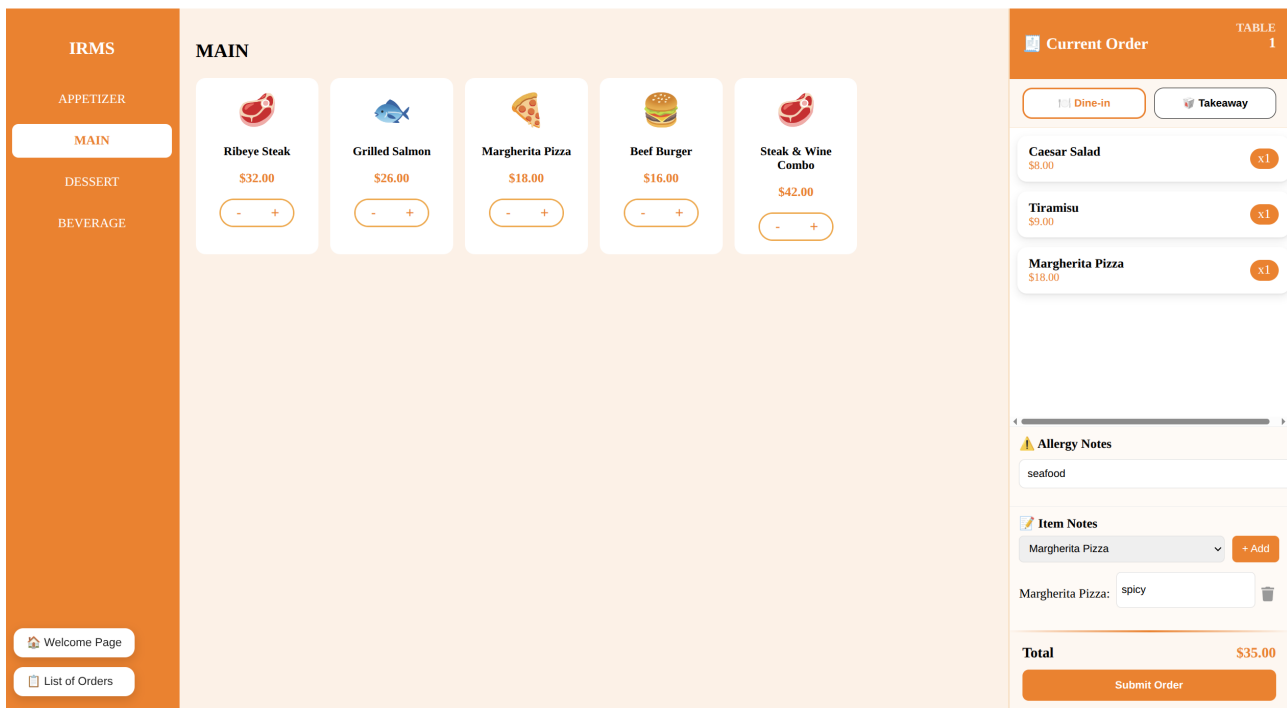


Figure 17: Ordering Page

There are 2 components in this page: **MenuList** and **OrderPanel**:

- **MenuList** component is responsible for displaying all available menu items to the staff and allowing item selection.
 - The component retrieves menu data through the utility function `initMenu()`, which asynchronously fetches data from two API endpoints: `/api/v1/menu/categories` and `/api/v1/menu/items`.
 - These two API calls are executed in parallel using `Promise.all`, improving performance by reducing total loading time.
 - After fetching, the data is stored in a module-level cache, ensuring that subsequent accesses do not trigger additional API requests.
 - The raw menu items are further processed by adding visual metadata (e.g., emoji icons) using the `pickEmoji()` function. This enhances the user interface without modifying backend data.
 - Once initialization is complete, the component accesses the cached data through the synchronous function `getMenu()`, which returns both categories and menu items.
 - The UI then renders menu items grouped by categories, allowing staff to easily browse and select items.
 - When a menu item is selected, the component triggers order-related actions, including: increase or decrease the quantity through the global state management system.
- **OrderPanel**

The **OrderPanel** component is responsible for displaying all selected dishes and their corresponding quantities. In addition, it allows staff to provide additional information such as allergy notes and custom notes for specific dishes before submitting the order.

- The component supports two main workflows:
 - * **Create a new order:** When accessing the route `/create-order?tableId={table_id}`, the system initializes an empty order state, where no items or notes are pre-filled.
 - * **Modify an existing order:** When accessing the route `/create-order?tableId={table_id}&edit={order_id}`, the system retrieves the existing order data using the function `getOrderById(orderId)`, which sends a request to the API endpoint `/api/v1/orders/{order_id}`. The returned data is then used to populate the current order state for editing.
- The component allows staff to:

- * Add customization options for each item.
- * Provide allergy notes and special instructions for individual dishes.
- When the user clicks **Submit Order** or **Update Order**, the system invokes the `submitOrder()` function to process the request.
- The submission process includes multiple steps:
 - * If creating a new order, the system first sends a POST request to `/api/v1/orders` to create a new order and obtain its `id`.
 - * For both new and existing orders, each selected item is sent individually via POST requests to `/api/v1/orders/{order_id}/items`, including quantity, customization data, and allergy notes.
 - * After all items are added, the system finalizes the order by sending a POST request to `/api/v1/orders/{order_id}/submit`.

Next, the system directs the user to the List of Orders page with route `/orders?tableId=${table.id}`. The purpose of passing the `table_id` parameter is to display only the orders associated with the selected table. If no parameter is provided, the system defaults to displaying orders of table 1.

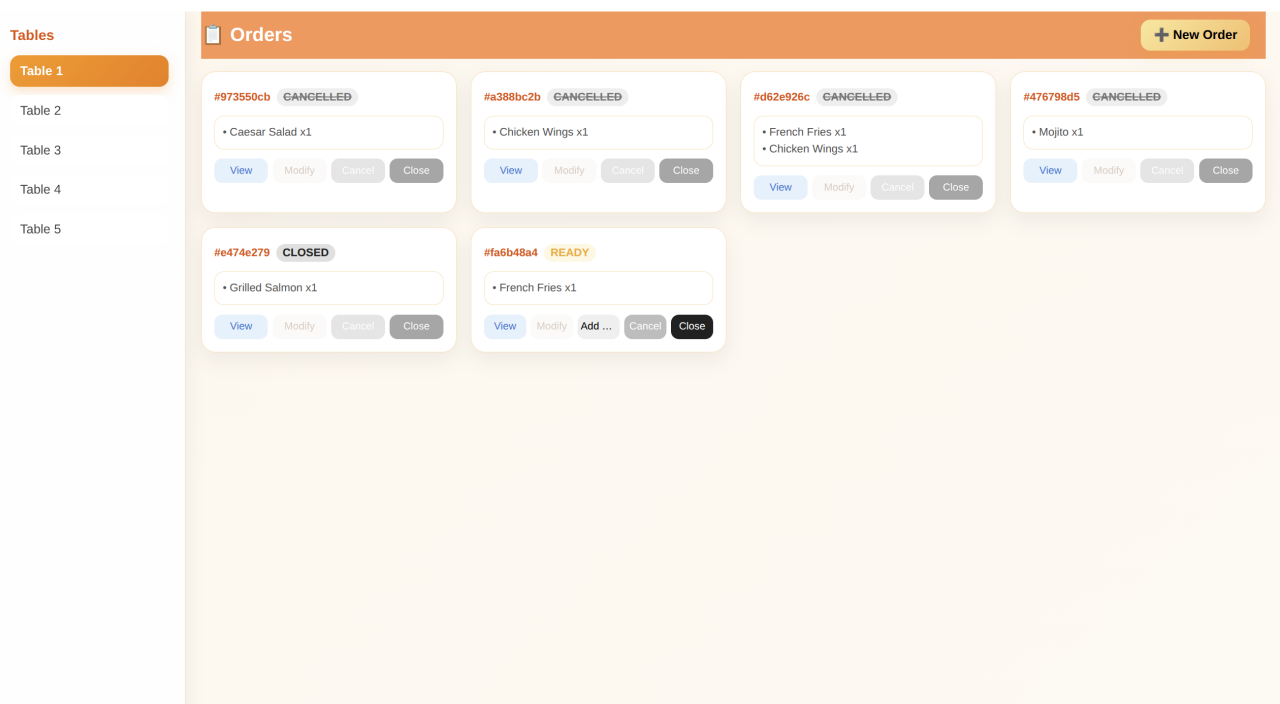


Figure 18: List of Orders Page



Figure 19: List tables are occupied

- The system retrieves order data using the function `getOrders()`, which sends a request to the API endpoint `/api/v1/orders`. When a `table_id` is provided, the request is filtered using the query parameter `?table_id={table_id}` to return only relevant orders.
- Each order card on the UI provides four main actions:
 - **View:** Retrieves detailed information of a specific order using the function `getOrderById(orderId)`, which calls the API endpoint `/api/v1/orders/{order_id}`. The detailed data is then displayed to the user.
 - **Modify:** Navigates the user to the order creation page with edit mode enabled using the route `/create-order?tableId={table_id}&edit={order_id}`. The existing order data is loaded and can be modified.
 - **Cancel:** Sends a `POST` request to the endpoint `/api/v1/orders/{order_id}/cancel` via the function `cancelOrder(orderId)` to cancel the selected order.
 - **Close:** Completes the order lifecycle by first sending a `POST` request to `/api/v1/orders/{order_id}/serve` (if not already served), followed by a `POST` request to `/api/v1/orders/{order_id}/close` using the function `closeOrder(orderId)`. This transitions the order to a finalized state.

3.4.2 KDS Feature

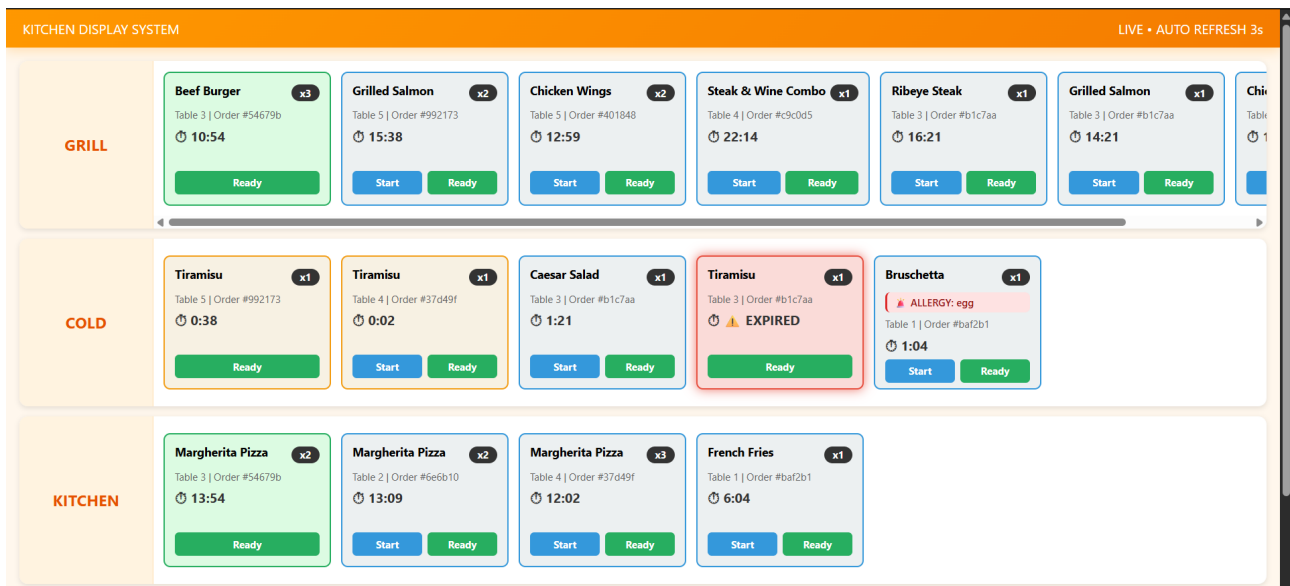


Figure 20: Kitchen Display System (KDS) Page

The Kitchen Display System (KDS) provides real-time visibility of active orders for kitchen staff. It aggregates order items by preparation stations and displays them as individual dish cards, allowing staff to monitor progress and manage cooking workflows efficiently.

• KDS Page Initialization

The KDS interface is implemented in `KDSPage.jsx`, which serves as the main entry point of the feature (`/kds`).

- The system retrieves station data from the backend via the API endpoint: `/api/v1/kds/stations`.
- For each station, active tickets are fetched using:
`/api/v1/kds/stations/{station_id}/tickets?status=ACTIVE`.
- These requests are executed asynchronously using `Promise.all`, ensuring efficient parallel data fetching.
- The retrieved data is transformed into a mapping structure (`ticketsMap`), where each station ID is associated with its corresponding list of tickets.
- The system also fetches menu item data from `/api/v1/menu` and stores it in a local map (`menuItems`) for quick lookup.
- To simulate near real-time updates, the data fetching process is executed periodically every 3 seconds using `setInterval`.

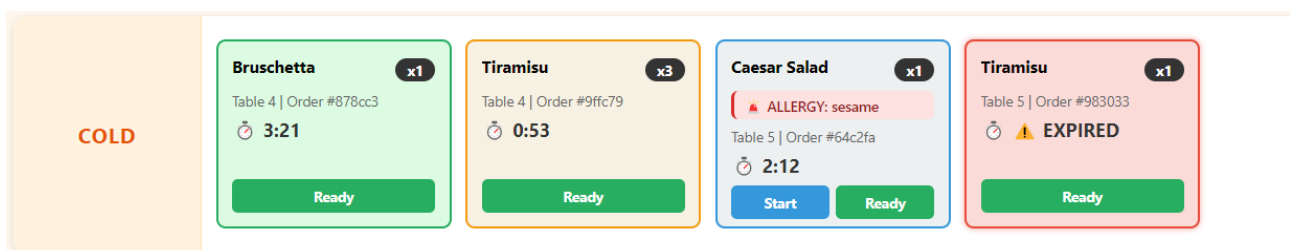


Figure 21: Station Row with Dish Cards

• Station-Based Rendering

The `StationRow` component is responsible for rendering each kitchen station.

- Each station displays its name using the `StationHeader` component.

- Instead of rendering tickets directly, the system transforms ticket data into individual dish items.
- Each ticket may contain multiple items; therefore, the system flattens the ticket structure into a list of dishes using `flatMap()`.
- Additional metadata such as `table_number`, `order_id`, and `placed_at` is attached to each dish to support display and timing logic.
- The resulting dish list is rendered as a horizontal collection of `DishCard` components.

• Dish Card and Timer Logic

Each dish is represented by a `DishCard` component, which encapsulates both UI and behavior.

- The component retrieves menu information (e.g., name and preparation time) using the `menuItems` map.
- A custom hook `useDishTimer` is used to calculate:
 - * Remaining preparation time
 - * Expiration state when the dish exceeds its expected preparation time
- The timer is updated every second using `setInterval`, ensuring real-time countdown behavior.
- Visual indicators are applied based on timing and status conditions:
 - * **QUEUED**: dish is waiting to be prepared
 - * **PREPARING**: dish is being cooked
 - * **WARNING**: remaining time is below threshold
 - * **EXPIRED**: dish exceeds preparation time
- When the timer reaches zero, the system triggers an alert sound using the `useSoundAlert` hook.

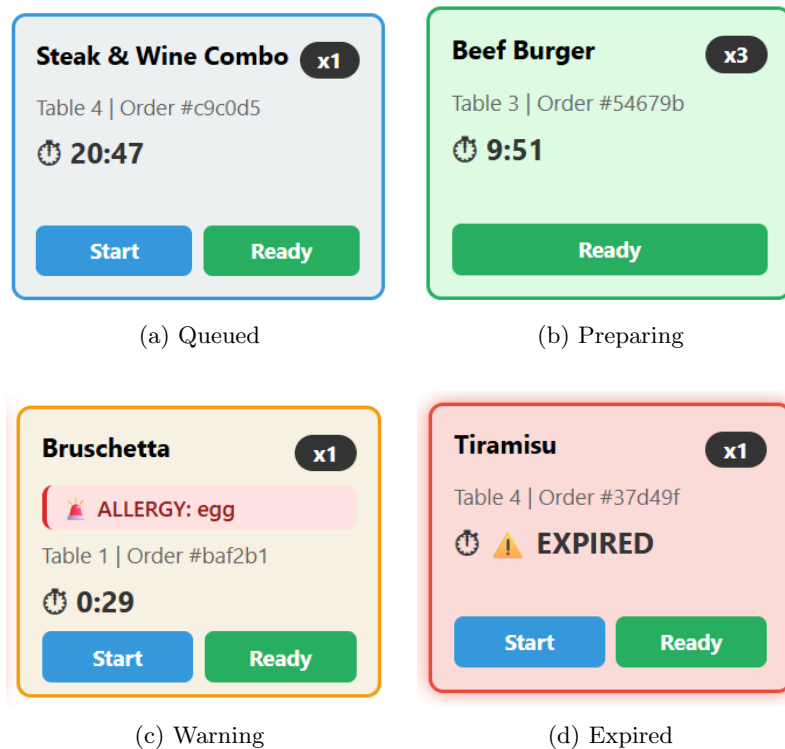


Figure 22: Visual States of DishCard in the Kitchen Display System

The top row represents normal preparation flow, while the bottom row highlights critical timing states that require immediate attention from kitchen staff.

• Sound Alert Mechanism

The alert sound is implemented using the Web Audio API via the `useSoundAlert` hook.

- The hook creates an `AudioContext` and generates a sine wave using an oscillator.

- The sound is configured with:
 - * Frequency: 1000 Hz
 - * Duration: 0.5 seconds
 - * Gradual volume decay using exponential ramp
- The alert is triggered automatically when a dish becomes overdue.
- Due to browser autoplay restrictions, the audio context is created and resumed only after a user interaction (e.g., click event).

- **Dish Status Management**

The system supports workflow transitions for each dish using the `useDishStatus` hook.

- Staff can update dish status through UI buttons:
 - * **Start**: transitions a dish from `QUEUED` to `PREPARING`
 - * **Ready**: marks the dish as completed
- These actions trigger API calls to update the backend state.
- A temporary transitioning state is used to provide immediate UI feedback while awaiting server confirmation.

- **Summary of Workflow**

- The system continuously fetches active kitchen data.
- Orders are grouped by stations and decomposed into individual dishes.
- Each dish is tracked independently with a real-time timer.
- Visual cues and sound alerts assist kitchen staff in prioritizing tasks.
- Staff can interact with dishes directly to update their preparation status.

4 Apply SOLID Principles

The IRMS was designed around the five SOLID principles to produce a modular, testable codebase that is resilient to the requirement churn typical of restaurant operations software. Within the service-based architecture and its domain partitioning into Ordering, Menu, Table, KDS, Payment, Inventory, Analytics, and Administration services, the SOLID principles shape both the boundaries between services and the internal structure of each one. They are most clearly realised in the Ordering module – one of two services implemented in this iteration alongside KDS – which therefore serves as the worked example throughout this section.

This chapter is organised in three parts:

- Section 4.1 presents the architectural decisions that establish the conditions under which SOLID can be applied meaningfully.
- Section 4.2 walks through each of the five principles in turn, explaining how each is realised across those decisions.
- Section 4.3 closes with implementation-level evidence demonstrating that the principles are enforced rather than aspirational.

A reflection on the overall discipline appears in Section 4.4. The Ordering class diagram (Figure 2) accompanies the discussion as a visual companion.

4.1 Architecture-Design Level

Three foundational design decisions in IRMS create the conditions under which SOLID can be applied substantively rather than ornamentally. Each decision is independent of any single principle but underwrites several at once.

Actor-aligned service partitioning. The back end is partitioned into eight domain services, each owned by a single restaurant role whose workflow drives change requests against that service.

- **OrderService** – answers to the server taking and submitting orders.
- **MenuService** – answers to the menu manager curating dishes and prices.

- **TableService** – answers to the host seating guests.
- **KdsService** – answers to the chef working the kitchen display.
- **PaymentService**, **InventoryService**, **AnalyticsService**, **AdministrationService** – each map to a distinct actor (cashier, stock manager, owner, system administrator) and are scaffolded for later iterations.

Because every service is owned by a single actor, change requests from one role cannot ripple sideways into another role’s logic. The trade-off is that any cross-domain workflow – for example, the server’s place-order path that touches menu availability and table occupancy – must coordinate through explicit service contracts rather than reaching into shared internals.

Protocol-based persistence boundaries. Every domain service depends on its persistence layer through a narrow abstract contract rather than a concrete database class. Four such contracts exist in the current iteration:

- **OrderRepository**, **MenuRepository**, **TableRepository**, and **StationRepository**, each colocated with its owning module.
- Each contract exposes only the methods its module legitimately requires.
- Contracts are declared as Python **Protocols** – a structural-typing mechanism that permits concrete adapters to fulfil the contract by matching its method signatures alone, without explicit inheritance.

This is what makes the persistence layer migration-friendly: alternative adapters – in-memory fakes, caches, read replicas, remote-RPC stubs – can substitute behind the same contract, provided they honour both its structural shape and its behavioural semantics.

Typed event log on Ordering lifecycle transitions. Every state transition in the Ordering module appends an immutable row to a dedicated **order_event** table, classified by a small enumeration of event types: **PLACED**, **STATUS_CHANGED**, **CANCELLED**, **RECALLED**, **ROUTED**.

- The log is an append-only stream that future consumers can attach to without modification to the producing service.
- Candidate consumers include analytics dashboards, push-notification dispatchers, and compliance audit pipelines.
- Today the stream is queryable as an audit trail; tomorrow it is the publish-subscribe substrate for cross-module integration.
- The same pattern is the planned extension hook for Payment, Inventory, and Analytics modules in subsequent iterations.

These three decisions are interdependent. Actor-aligned partitioning gives each principle a clean unit of responsibility to work over; Protocol-based boundaries give each principle a well-defined contract surface to act on; the event log gives the producer side a way to admit new consumers without breaking. Without all three, the SOLID arguments that follow would have nowhere to land.

4.2 Five Principles Applied

Single Responsibility Principle. A module should be responsible to one and only one actor, where an actor is a role whose workflow drives change requests.

- The actor-aligned partitioning realises this principle at the service level: each of the eight domain services exists to serve one role.
- A change request from the menu manager modifies the Menu module without rippling into the server’s, host’s, or chef’s modules.
- *Deliberate non-split:* **OrderService** retains three chef-facing methods (**update_item_status**, **transition_order_to_ready**, **recall_order**) rather than relocating them to **KdsService**, because the order aggregate is the single source of truth for item state.
- **KdsService** composes **OrderService** as a dependency rather than duplicating the state machine, keeping the chef’s workflow cohesive without fracturing order invariants across two modules.

Open/Closed Principle. Modules should be open for extension and closed for modification. In IRMS, OCP applies to the consumer side of the order lifecycle, not the producer side.

- *Open for extension.* The event log establishes a stable extension seam at which new consumers can attach without any modification to the services that emit events. A future analytics dashboard, notification channel, or audit hook can each be added by reading the existing stream, with no edit to **OrderService**.

- *Closed for modification.* The producer side – the lifecycle states themselves – is intentionally closed. Order states are business-defined and finite, and adding a new lifecycle state is the kind of change that should require deliberate code review rather than a configuration-style extension.
- *Architectural commitment.* The OCP boundary sits at consumer attachment; producer behaviour remains governed by review.

Liskov Substitution Principle. If S is a subtype of T , instances of T may be replaced with instances of S without altering correctness. The substitute must preserve the supertype’s preconditions, postconditions, and invariants – not merely match its method shapes.

- The Protocol-based persistence boundaries provide the supertype contracts; concrete adapters substitute behind them.
- *Important qualification.* A structural Protocol verifies that method signatures align, but it cannot prove that a substitute honours the contract’s semantics – for example, that saving an order is idempotent on the same identifier, or that listing events returns them in insertion order.
- Behavioural conformance therefore requires runtime evidence, supplied by the test suite (Section 4.3).
- The architectural value of this design is migration-friendliness: future cache, read-replica, or remote-RPC adapters can substitute behind the same contracts, conditional on honouring both their structural shape and their behavioural semantics.

Interface Segregation Principle. Clients should not be forced to depend on methods they do not use. The four narrow Repository contracts realise this principle directly.

- Each contract is small enough to read at a glance, exposes only the operations its single owning module needs, and never accretes methods that exist to serve a different consumer.
- *Rejected alternative.* An earlier draft considered an omnibus persistence contract covering every concern in the system. This was rejected on ISP grounds: a consumer that only needs to read tables would have ended up transitively bound to order persistence, menu mutation, and station bookkeeping simply because those methods sat on the same contract.
- *Trade-off.* One extra contract declaration per persistence concern, paid back every time a consumer is spared methods it does not use.
- *Composition is not violation.* Where `OrderService` legitimately requires multiple contracts (the order, menu, and table boundaries), it composes them in its constructor as separate dependencies. Each contract stays narrow on its own; the service genuinely depends on every method it imports.
- Reducing the dependency count would force either an SRP violation (merging unrelated responsibilities into a single dependency) or a fresh ISP violation (hiding the dependencies behind a Facade that aggregates methods callers do not all use).

Dependency Inversion Principle. High-level modules should not depend on low-level modules; both should depend on abstractions. In IRMS, DIP is applied at two seams with deliberately different strictness.

- *Persistence boundary – strict.* Every domain service depends on a Repository contract, never on a concrete database adapter. The contract is owned by the domain layer; the adapter is owned by the infrastructure layer; the dependency arrow points from infrastructure to domain.
- *Cross-service seam – relaxed.* For example, `OrderService` calling `MenuService` and `TableService`. Under the cross-service rule established in Section 3.1, the published API surface of a peer service is treated as its abstraction, and `OrderService` receives concrete `MenuService` and `TableService` via constructor injection rather than via extracted Port Protocols.

The relaxation at the cross-service seam is justified on three grounds:

- The peer services are owned by the same team and live in the same deployable, so the boundary they cross is logical rather than organisational or process-level.
- No second implementation of either service is foreseen within the project’s scope.
- Should that scope ever change, extracting a `MenuPort` or `TablePort` Protocol from the existing service is a mechanical refactor that the Protocol-based design itself accommodates without disrupting callers.

DIP underpins the other four principles: once persistence boundaries became contracts, OCP, LSP, and ISP followed with minimal additional effort, because the abstraction surfaces required to satisfy them were already in place.

4.3 Implementation Level

The architecture-level claims of Section 4.2 are backed by concrete, verifiable structure in the codebase. This section walks the same five principles in the same order, this time showing the implementation evidence that enforces each one.

Single Responsibility Principle – one class, one file, one responsibility. Each domain service occupies its own Python package, with the business logic, the persistence contract, and the SQLAlchemy adapter held in separate files within the package.

- **OrderService** lives alone in its file and exposes only methods belonging to the server-and-chef workflow.
- **MenuService** contains only category-and-item curation operations.
- **TableService** is a deliberately thin read-only collaborator.
- A repository-wide search for SQLAlchemy imports inside the service layer returns no matches: the persistence framework appears only inside adapter files and the composition root, never inside the domain logic that depends on the contracts.

Open/Closed Principle – uniform event-publication discipline. The event log is realised as a method on the order repository contract that appends a typed row to the `order_event` table.

- Every lifecycle transition in **OrderService** terminates with a call to this method before returning.
- Submitting an order publishes a **PLACED** event; cancelling publishes a **CANCELLED** event; serving publishes a **STATUS_CHANGED** event with a **SERVED** payload; the three chef-facing transitions publish their respective events.
- The call sites are few in number, identical in shape, and routed through a single repository method.
- A future migration from database-row pub-sub to in-process event-bus dispatch is therefore a single edit at that repository method, with no change required at any of the call sites.
- An in-process event bus class already exists in the shared library as forward-compatibility scaffolding rather than dead code; it is the prepared migration target, and both approaches share the same call site by design.

Liskov Substitution Principle – substitution observable through a single composition root. The composition root is the only file in the codebase where contracts and concrete adapters meet, and it is what makes Liskov substitution observable rather than aspirational.

- Everywhere else in the domain layer sees the contract type alone; only the composition root imports both sides.
- FastAPI's dependency injection plays the role of the Abstract Factory: a request handler asks for an **OrderRepository**, the factory constructs the SQLAlchemy adapter bound to the current request's database session, and the handler receives an abstraction whose concrete identity it never needs to know.
- Production code receives the SQL adapters; the test suite injects in-memory adapters into the exact same service constructors.
- The two adapter families pass the same suite of behavioural tests against the same service code, with only the persistence backend differing.
- This is the empirical evidence the structural Protocol cannot supply on its own – evidence under the behaviours actually exercised by the domain services that substitution preserves the contracts on which those services depend.
- *Bounded scope.* Only two adapter families exist today, so richer substitution scenarios such as read replicas or remote-RPC fakes are accommodated by the design but not yet demonstrated.

Interface Segregation Principle – bounded contract size. Each Protocol carries a small, fixed method set corresponding exactly to what its single consumer requires – no more, no less.

- The menu contract covers category and item lookup and persistence.
- The order contract covers listing, retrieval, persistence, and event append.
- The table contract covers listing and lookup.
- The station contract covers listing, lookup, and persistence.

- The contract files can be scanned in under a minute, and a reviewer can verify that every method on every contract is used by every caller of that contract.
- *Documented next step.* Should query operations grow further with filters, projections, or history scans, the order contract will be split into reader and writer halves along CQRS lines so that read-heavy clients no longer depend on write methods.

Dependency Inversion Principle – contracts in the domain layer, adapters at the boundary. The composition root described under LSP also realises DIP at the implementation level, but the strictness of the inversion differs across the two seams identified in Section 4.2.

- *Persistence boundary – strict.* Domain services type-annotate their constructor parameters with the Repository contract. The concrete SQLAlchemy adapter is constructed only inside the composition root and upcast to the contract type before injection. No domain service imports any persistence-layer module.
- *Cross-service seam – relaxed.* `OrderService` accepts `MenuService` and `TableService` as concrete classes via constructor injection rather than through extracted Port Protocols. Under the cross-service rule established in Section 3.1, the public method surface of each peer service is treated as its abstraction.
- Storage technology can be swapped without touching a single line of domain logic – the test suite is the standing demonstration, swapping the adapter family while leaving `OrderService` unchanged.
- *Reversibility.* If a second implementation of `MenuService` or `TableService` ever becomes necessary, extracting a Port Protocol is a mechanical refactor: the constructor signatures in callers are already shaped to receive an abstraction.

Taken together, these five implementation patterns – single-class files, uniform event publication, a single composition root, bounded contract sizes, and Protocol-typed dependencies at the boundary – ensure that the SOLID claims of Section 4.2 are enforceable code structure rather than architectural aspiration. A reviewer can verify each claim by reading the contracts, walking the constructors, and running the test suite; no claim in this chapter rests on documentation alone.

4.4 Reflection Report

The five principles converge into a single coherent set of structural choices rather than five independent edits: actor-aligned services depend on narrow contracts that are realised by substitutable adapters, all wired at a single composition root, while a domain-event log opens the door to new consumers without touching existing producers. Each principle reinforces the others, and removing any one would erode the leverage of the remaining four.

Three improvements followed directly from this discipline:

- Splitting Ordering responsibility along actor lines eliminated the temptation to scatter menu-availability, table-occupancy, and order-lifecycle logic into a single class.
- Defining repository abstractions and injecting them at the boundary broke the hard coupling between domain services and the persistence stack. Without that step, every order-entry method would have ended up doing both business validation and database row construction in the same place.
- Routing Ordering-to-KDS communication through a published API contract – combined with the prepared event-log seam – prevented the kind of internals-coupling that would have made every Ordering change a KDS regression risk.

The discipline also surfaced challenges that required deliberate design judgement rather than reflexive application of the principles:

- *An SRP boundary question.* Should chef-facing transitions like `update_item_status` live in `KdsService`, where they are invoked, or in `OrderService`, where the state they mutate is owned? Both placements look defensible at first glance, but only the latter preserves the order aggregate as the single source of truth for item state, so the design accepts the slight asymmetry in favour of invariant integrity.
- *An OCP scope question.* A strict reading would extend OCP to lifecycle states themselves, but lifecycle states are business-defined, finite, and their changes are precisely the kind of edit that should require code review rather than a configuration-style extension. The OCP line was therefore drawn at consumer attachment, not at producer behaviour.
- *A DIP strictness question* at the cross-service seam, resolved in favour of pragmatic injection of concrete services rather than proliferating Port contracts for which no second implementation is foreseen.

Two costs were accepted as trade-offs the design deliberately did not pay:

- The order state machine is not OCP-extensible: a new lifecycle state remains an `OrderService` edit, by design.
- Cross-service injection uses concrete service classes rather than extracted Port contracts, a relaxation of strict Martin DIP justified by the cross-service design rule of Section 3.1.

Both concessions are reversible: should the project ever require a second implementation of either kind, the Protocol-based design accommodates the extraction without restructuring callers.

The principle that paid back the most was Dependency Inversion. Once persistence boundaries were Protocols, every other SOLID outcome became a cheap follow-on rather than an independent design battle. The Ordering class diagram (Figure 2) visualises this structure and lets a reader verify each principle without reading code.